Coroutine Theory

lewissbaker.github.io/2017/09/25/coroutine-theory

Asymmetric Transfer 25 September 2017

Sep 25, 2017

This is the first of a series of posts on the C++ Coroutines TS, a new language feature that is currently on track for inclusion into the C++20 language standard.

In this series I will cover how the underlying mechanics of C++ Coroutines work as well as show how they can be used to build useful higher-level abstractions such as those provided by the cppcoro library.

In this post I will describe the differences between functions and coroutines and provide a bit of theory about the operations they support. The aim of this post is introduce some foundational concepts that will help frame the way you think about C++ Coroutines.

Coroutines are Functions are Coroutines

A coroutine is a generalisation of a function that allows the function to be suspended and then later resumed.

I will explain what this means in a bit more detail, but before I do I want to first review how a “normal” C++ function works.

“Normal” Functions

A normal function can be thought of as having two operations: **Call** and **Return** (Note that I’m lumping “throwing an exception” here broadly under the **Return** operation).

The **Call** operation creates an activation frame, suspends execution of the calling function and transfers execution to the start of the function being called.

The **Return** operation passes the return-value to the caller, destroys the activation frame and then resumes execution of the caller just after the point at which it called the function.

Let’s analyse these semantics a little more...

Activation Frames

So what is this ‘activation frame’ thing?

You can think of the activation frame as the block of memory that holds the current state of a particular invocation of a function. This state includes the values of any parameters that were passed to it and the values of any local variables.

For “normal” functions, the activation frame also includes the return-address - the address of the instruction to transfer execution to upon returning from the function - and the address of the activation frame for the invocation of the calling function. You can think of these pieces of information together as describing the ‘continuation’ of the function-call. ie. they describe which invocation of which function should continue executing at which point when this function completes.

With “normal” functions, all activation frames have strictly nested lifetimes. This strict nesting allows use of a highly efficient memory allocation data-structure for allocating and freeing the activation frames for each of the function calls. This data-structure is commonly referred to as “the stack”.

When an activation frame is allocated on this stack data structure it is often called a “stack frame”.

This stack data-structure is so common that most (all?) CPU architectures have a dedicated register for holding a pointer to the top of the stack (eg. in X64 it is the `rsp` register).

To allocate space for a new activation frame, you just increment this register by the frame-size. To free space for an activation frame, you just decrement this register by the frame-size.

The ‘Call’ Operation

When a function calls another function, the caller must first prepare itself for suspension.

This ‘suspend’ step typically involves saving to memory any values that are currently held in CPU registers so that those values can later be restored if required when the function resumes execution. Depending on the calling convention of the function, the caller and callee may coordinate on who saves these register values, but you can still think of them as being performed as part of the **Call** operation.

The caller also stores the values of any parameters passed to the called function into the new activation frame where they can be accessed by the function.

Finally, the caller writes the address of the resumption-point of the caller to the new activation frame and transfers execution to the start of the called function.

In the X86/X64 architecture this final operation has its own instruction, the `call` instruction, that writes the address of the next instruction onto the stack, increments the stack register by the size of the address and then jumps to the address specified in the instruction’s operand.

The ‘Return’ Operation

When a function returns via a `return` -statement, the function first stores the return value (if any) where the caller can access it. This could either be in the caller’s activation frame or the function’s activation frame (the distinction can get a bit blurry for parameters and return values that cross the boundary between two activation frames).

Then the function destroys the activation frame by:

- Destroying any local variables in-scope at the return-point.
- Destroying any parameter objects

- Freeing memory used by the activation-frame

And finally, it resumes execution of the caller by:

- Restoring the activation frame of the caller by setting the stack register to point to the activation frame of the caller and restoring any registers that might have been clobbered by the function.
- Jumping to the resume-point of the caller that was stored during the 'Call' operation.

Note that as with the 'Call' operation, some calling conventions may split the repsonsibilities of the 'Return' operation across both the caller and callee function's instructions.

Coroutines

Coroutines generalise the operations of a function by separating out some of the steps performed in the **Call** and **Return** operations into three extra operations: **Suspend**, **Resume** and **Destroy**.

The **Suspend** operation suspends execution of the coroutine at the current point within the function and transfers execution back to the caller or resumer without destroying the activation frame. Any objects in-scope at the point of suspension remain alive after the coroutine execution is suspended.

Note that, like the **Return** operation of a function, a coroutine can only be suspended from within the coroutine itself at well-defined suspend-points.

The **Resume** operation resumes execution of a suspended coroutine at the point at which it was suspended. This reactivates the coroutine's activation frame.

The **Destroy** operation destroys the activation frame without resuming execution of the coroutine. Any objects that were in-scope at the suspend point will be destroyed. Memory used to store the activation frame is freed.

Coroutine activation frames

Since coroutines can be suspended without destroying the activation frame, we can no longer guarantee that activation frame lifetimes will be strictly nested. This means that activation frames cannot in general be allocated using a stack data-structure and so may need to be stored on the heap instead.

There are some provisions in the C++ Coroutines TS to allow the memory for the coroutine frame to be allocated from the activation frame of the caller if the compiler can prove that the lifetime of the coroutine is indeed strictly nested within the lifetime of the caller. This can avoid heap allocations in many cases provided you have a sufficiently smart compiler.

With coroutines there are some parts of the activation frame that need to be preserved across coroutine suspension and there are some parts that only need to be kept around while the coroutine is executing. For example, the lifetime of a variable with a scope that does not span any coroutine suspend-points can potentially be stored on the stack.

You can logically think of the activation frame of a coroutine as being comprised of two parts: the 'coroutine frame' and the 'stack frame'.

The ‘coroutine frame’ holds part of the coroutine’s activation frame that persists while the coroutine is suspended and the ‘stack frame’ part only exists while the coroutine is executing and is freed when the coroutine suspends and transfers execution back to the caller/resumer.

The ‘Suspend’ operation

The **Suspend** operation of a coroutine allows the coroutine to suspend execution in the middle of the function and transfer execution back to the caller or resumer of the coroutine.

There are certain points within the body of a coroutine that are designated as suspend-points. In the C++ Coroutines TS, these suspend-points are identified by usages of the `co_await` or `co_yield` keywords.

When a coroutine hits one of these suspend-points it first prepares the coroutine for resumption by:

- Ensuring any values held in registers are written to the coroutine frame
- Writing a value to the coroutine frame that indicates which suspend-point the coroutine is being suspended at. This allows a subsequent **Resume** operation to know where to resume execution of the coroutine or so a subsequent **Destroy** to know what values were in-scope and need to be destroyed.

Once the coroutine has been prepared for resumption, the coroutine is considered ‘suspended’.

The coroutine then has the opportunity to execute some additional logic before execution is transferred back to the caller/resumer. This additional logic is given access to a handle to the coroutine-frame that can be used to later resume or destroy it.

This ability to execute logic after the coroutine enters the ‘suspended’ state allows the coroutine to be scheduled for resumption without the need for synchronisation that would otherwise be required if the coroutine was scheduled for resumption prior to entering the ‘suspended’ state due to the potential for suspension and resumption of the coroutine to race. I’ll go into this in more detail in future posts.

The coroutine can then choose to either immediately resume/continue execution of the coroutine or can choose to transfer execution back to the caller/resumer.

If execution is transferred to the caller/resumer the stack-frame part of the coroutine’s activation frame is freed and popped off the stack.

The ‘Resume’ operation

The **Resume** operation can be performed on a coroutine that is currently in the ‘suspended’ state.

When a function wants to resume a coroutine it needs to effectively ‘call’ into the middle of a particular invocation of the function. The way the resumer identifies the particular invocation to resume is by calling the `void resume()` method on the coroutine-frame handle provided to the corresponding **Suspend** operation.

Just like a normal function call, this call to `resume()` will allocate a new stack-frame and store the return-address of the caller in the stack-frame before transferring execution to the function.

However, instead of transferring execution to the start of the function it will transfer execution to the point in the function at which it was last suspended. It does this by loading the resume-point from the coroutine-frame and jumping to that point.

When the coroutine next suspends or runs to completion this call to `resume()` will return and resume execution of the calling function.

The 'Destroy' operation

The **Destroy** operation destroys the coroutine frame without resuming execution of the coroutine.

This operation can only be performed on a suspended coroutine.

The **Destroy** operation acts much like the **Resume** operation in that it re-activates the coroutine's activation frame, including allocating a new stack-frame and storing the return-address of the caller of the **Destroy** operation.

However, instead of transferring execution to the coroutine body at the last suspend-point it instead transfers execution to an alternative code-path that calls the destructors of all local variables in-scope at the suspend-point before then freeing the memory used by the coroutine frame.

Similar to the **Resume** operation, the **Destroy** operation identifies the particular activation-frame to destroy by calling the `void destroy()` method on the coroutine-frame handle provided during the corresponding **Suspend** operation.

The 'Call' operation of a coroutine

The **Call** operation of a coroutine is much the same as the call operation of a normal function. In fact, from the perspective of the caller there is no difference.

However, rather than execution only returning to the caller when the function has run to completion, with a coroutine the call operation will instead resume execution of the caller when the coroutine reaches its first suspend-point.

When performing the **Call** operation on a coroutine, the caller allocates a new stack-frame, writes the parameters to the stack-frame, writes the return-address to the stack-frame and transfers execution to the coroutine. This is exactly the same as calling a normal function.

The first thing the coroutine does is then allocate a coroutine-frame on the heap and copy/move the parameters from the stack-frame into the coroutine-frame so that the lifetime of the parameters extends beyond the first suspend-point.

The 'Return' operation of a coroutine

The **Return** operation of a coroutine is a little different from that of a normal function.

When a coroutine executes a `return` -statement (`co_return` according to the TS) operation it stores the return-value somewhere (exactly where this is stored can be customised by the coroutine) and then destructs any in-scope local variables (but not parameters).

The coroutine then has the opportunity to execute some additional logic before transferring execution back to the caller/resumer.

This additional logic might perform some operation to publish the return value, or it might resume another coroutine that was waiting for the result. It's completely customisable.

The coroutine then performs either a **Suspend** operation (keeping the coroutine-frame alive) or a **Destroy** operation (destroying the coroutine-frame).

Execution is then transferred back to the caller/resumer as per the **Suspend**/**Destroy** operation semantics, popping the stack-frame component of the activation-frame off the stack.

It is important to note that the return-value passed to the **Return** operation is not the same as the return-value returned from a **Call** operation as the return operation may be executed long after the caller resumed from the initial **Call** operation.

An illustration

To help put these concepts into pictures, I want to walk through a simple example of what happens when a coroutine is called, suspends and is later resumed.

So let's say we have a function (or coroutine), `f()` that calls a coroutine, `x(int a)` .

Before the call we have a situation that looks a bit like this:

```
STACK                     REGISTERS               HEAP

                          +------+
+---------------+ <------ | rsp  |
|  f()          |         +------+
+---------------+
| ...           |
|               |
```

Then when `x(42)` is called, it first creates a stack frame for `x()` , as with normal functions.

```
STACK                     REGISTERS               HEAP
+----------------+ <-+
|  x()           |   |
| a  = 42        |   |
| ret= f()+0x123 |   |    +------+
+----------------+   +--- | rsp  |
|  f()           |        +------+
+----------------+
| ...            |
|                |
```

Then, once the coroutine `x()` has allocated memory for the coroutine frame on the heap and copied/moved parameter values into the coroutine frame we'll end up with something that looks like the next diagram. Note that the compiler will typically hold the address of the coroutine frame in a separate register to the stack pointer (eg. MSVC stores this in the `rbp` register).

```
STACK                     REGISTERS              HEAP
+----------------+ <-+
|  x()           |   |
| a  = 42        |   |                  +-->  +-----------+
| ret= f()+0x123 |   |    +------+       |     |  x()      |
+----------------+   +--- | rsp  |       |     | a =  42   |
|  f()           |        +------+       |     +-----------+
+----------------+        | rbp  | ------+
| ...            |        +------+
|                |
```

If the coroutine `x()` then calls another normal function `g()` it will look something like this.

```
STACK                     REGISTERS              HEAP
+----------------+ <-+
|  g()           |   |
| ret= x()+0x45  |   |
+----------------+   |
|  x()           |   |
| coroframe      | --|-------------------+
| a  = 42        |   |                  +-->  +-----------+
| ret= f()+0x123 |   |    +------+            |  x()      |
+----------------+   +--- | rsp  |            | a =  42   |
|  f()           |        +------+            +-----------+
+----------------+        | rbp  |
| ...            |        +------+
|                |
```

When `g()` returns it will destroy its activation frame and restore `x()`'s activation frame. Let's say we save `g()`'s return value in a local variable `b` which is stored in the coroutine frame.

```
STACK                     REGISTERS              HEAP
+----------------+ <-+
|  x()           |   |
| a  = 42        |   |                  +-->  +-----------+
| ret= f()+0x123 |   |    +------+       |     |  x()      |
+----------------+   +--- | rsp  |       |     | a =  42   |
|  f()           |        +------+       |     | b = 789   |
+----------------+        | rbp  | ------+     +-----------+
| ...            |        +------+
|                |
```

If `x()` now hits a suspend-point and suspends execution without destroying its activation frame then execution returns to `f()`.

This results in the stack-frame part of `x()` being popped off the stack while leaving the coroutine-frame on the heap. When the coroutine suspends for the first time, a return-value is returned to the caller. This return value often holds a handle to the coroutine-frame that suspended that can be used to

later resume it. When `x()` suspends it also stores the address of the resumption-point of `x()` in the coroutine frame (call it `RP` for resume-point).

```
STACK                     REGISTERS               HEAP
                                        +----> +-----------+
                          +------+      |      |  x()      |
+----------------+ <----- | rsp  |      |      | a =  42   |
|  f()           |        +------+      |      | b = 789   |
| handle     ----|---+    | rbp  |      |      | RP=x()+99 |
| ...            |   |    +------+      |      +-----------+
|                |   |                  |
|                |   +------------------+
```

This handle may now be passed around as a normal value between functions. At some point later, potentially from a different call-stack or even on a different thread, something (say, `h()`) will decide to resume execution of that coroutine. For example, when an async I/O operation completes.

The function that resumes the coroutine calls a `void resume(handle)` function to resume execution of the coroutine. To the caller, this looks just like any other normal call to a `void` -returning function with a single argument.

This creates a new stack-frame that records the return-address of the caller to `resume()` , activates the coroutine-frame by loading its address into a register and resumes execution of `x()` at the resume-point stored in the coroutine-frame.

```
STACK                     REGISTERS               HEAP
+----------------+ <-+
|  x()           |   |                   +-->  +-----------+
| ret= h()+0x87  |   |    +------+       |     |  x()      |
+----------------+   +--- | rsp  |       |     | a =  42   |
|  h()           |        +------+       |     | b = 789   |
| handle         |        | rbp  | ------+     +-----------+
+----------------+        +------+
| ...            |
|                |
```

In summary

I have described coroutines as being a generalisation of a function that has three additional operations - ‘Suspend’, ‘Resume’ and ‘Destroy’ - in addition to the ‘Call’ and ‘Return’ operations provided by “normal” functions.

I hope that this provides some useful mental framing for how to think of coroutines and their control-flow.

In the next post I will go through the mechanics of the C++ Coroutines TS language extensions and explain how the compiler translates code that you write into coroutines.

C++ Coroutines: Understanding operator co_await

lewissbaker.github.io/2017/11/17/understanding-operator-co-await

Asymmetric Transfer 17 November 2017

Nov 17, 2017

In the previous post on Coroutine Theory I described the high-level differences between functions and coroutines but without going into any detail on syntax and semantics of coroutines as described by the C++ Coroutines TS (N4680).

The key new facility that the Coroutines TS adds to the C++ language is the ability to suspend a coroutine, allowing it to be later resumed. The mechanism the TS provides for doing this is via the new `co_await` operator.

Understanding how the `co_await` operator works can help to demystify the behaviour of coroutines and how they are suspended and resumed. In this post I will be explaining the mechanics of the `co_await` operator and introduce the related **Awaitable** and **Awaiter** type concepts.

But before I dive into `co_await` I want to give a brief overview of the Coroutines TS to provide some context.

What does the Coroutines TS give us?

- Three new language keywords: `co_await` , `co_yield` and `co_return`
- Several new types in the `std::experimental` namespace:
  - `coroutine_handle<P>`
  - `coroutine_traits<Ts...>`
  - `suspend_always`
  - `suspend_never`
- A general mechanism that library writers can use to interact with coroutines and customise their behaviour.
- A language facility that makes writing asynchronous code a whole lot easier!

The facilities the C++ Coroutines TS provides in the language can be thought of as a *low-level assembly-language* for coroutines. These facilities can be difficult to use directly in a safe way and are mainly intended to be used by library-writers to build higher-level abstractions that application developers can work with safely.

The plan is to deliver these new low-level facilities into an upcoming language standard (hopefully C++20) along with some accompanying higher-level types in the standard library that wrap these low-level building-blocks and make coroutines more accessible in a safe way for application developers.

Compiler <-> Library interaction

Interestingly, the Coroutines TS does not actually define the semantics of a coroutine. It does not define how to produce the value returned to the caller. It does not define what to do with the return value passed to the `co_return` statement or how to handle an exception that propagates out of the coroutine. It does not define what thread the coroutine should be resumed on.

Instead, it specifies a general mechanism for library code to customise the behaviour of the coroutine by implementing types that conform to a specific interface. The compiler then generates code that calls methods on instances of types provided by the library. This approach is similar to the way that a library-writer can customise the behaviour of a range-based for-loop by defining the `begin()` / `end()` methods and an `iterator` type.

The fact that the Coroutines TS doesn't prescribe any particular semantics to the mechanics of a coroutine makes it a powerful tool. It allows library writers to define many different kinds of coroutines, for all sorts of different purposes.

For example, you can define a coroutine that produces a single value asynchronously, or a coroutine that produces a sequence of values lazily, or a coroutine that simplifies control-flow for consuming `optional<T>` values by early-exiting if a `nullopt` value is encountered.

There are two kinds of interfaces that are defined by the coroutines TS: The **Promise** interface and the **Awaitable** interface.

The **Promise** interface specifies methods for customising the behaviour of the coroutine itself. The library-writer is able to customise what happens when the coroutine is called, what happens when the coroutine returns (either by normal means or via an unhandled exception) and customise the behaviour of any `co_await` or `co_yield` expression within the coroutine.

The **Awaitable** interface specifies methods that control the semantics of a `co_await` expression. When a value is `co_await` ed, the code is translated into a series of calls to methods on the awaitable object that allow it to specify: whether to suspend the current coroutine, execute some logic after it has suspended to schedule the coroutine for later resumption, and execute some logic after the coroutine resumes to produce the result of the `co_await` expression.

I'll be covering details of the **Promise** interface in a future post, but for now let's look at the **Awaitable** interface.

Awaiters and Awaitables: Explaining `operator co_await`

The `co_await` operator is a new unary operator that can be applied to a value. For example: `co_await someValue` .

The `co_await` operator can only be used within the context of a coroutine. This is somewhat of a tautology though, since any function body containing use of the `co_await` operator, by definition, will be compiled as a coroutine.

A type that supports the `co_await` operator is called an **Awaitable** type.

Note that whether or not the `co_await` operator can be applied to a type can depend on the context in which the `co_await` expression appears. The promise type used for a coroutine can alter the meaning of a `co_await` expression within the coroutine via its `await_transform` method (more on this later).

To be more specific where required I like to use the term **Normally Awaitable** to describe a type that supports the `co_await` operator in a coroutine context whose promise type does not have an `await_transform` member. And I like to use the term **Contextually Awaitable** to describe a type that only supports the `co_await` operator in the context of certain types of coroutines due to the presence of an `await_transform` method in the coroutine's promise type. (I'm open to better suggestions for these names here...)

An **Awaiter** type is a type that implements the three special methods that are called as part of a `co_await` expression: `await_ready` , `await_suspend` and `await_resume` .

Note that I have shamelessly "borrowed" the term 'Awaiter' here from the C# `async` keyword's mechanics that is implemented in terms of a `GetAwaiter()` method which returns an object with an interface that is eerily similar to the C++ concept of an **Awaiter**. See this post for more details on C# awaiters.

Note that a type can be both an **Awaitable** type and an **Awaiter** type.

When the compiler sees a `co_await <expr>` expression there are actually a number of possible things it could be translated to depending on the types involved.

Obtaining the Awaiter

The first thing the compiler does is generate code to obtain the **Awaiter** object for the awaited value. There are a number of steps to obtaining the awaiter object which are set out in N4680 section 5.3.8(3).

Let's assume that the promise object for the awaiting coroutine has type, `P` , and that `promise` is an l-value reference to the promise object for the current coroutine.

If the promise type, `P` , has a member named `await_transform` then `<expr>` is first passed into a call to `promise.await_transform(<expr>)` to obtain the **Awaitable** value, `awaitable` . Otherwise, if the promise type does not have an `await_transform` member then we use the result of evaluating `<expr>` directly as the **Awaitable** object, `awaitable` .

Then, if the **Awaitable** object, `awaitable` , has an applicable `operator co_await()` overload then this is called to obtain the **Awaiter** object. Otherwise the object, `awaitable` , is used as the awaiter object.

If we were to encode these rules into the functions `get_awaitable()` and `get_awaiter()` , they might look something like this:

```
template<typename P, typename T>
decltype(auto) get_awaitable(P& promise, T&& expr)
{
  if constexpr (has_any_await_transform_member_v<P>)
    return promise.await_transform(static_cast<T&&>(expr));
  else
    return static_cast<T&&>(expr);
}

template<typename Awaitable>
decltype(auto) get_awaiter(Awaitable&& awaitable)
{
  if constexpr (has_member_operator_co_await_v<Awaitable>)
    return static_cast<Awaitable&&>(awaitable).operator co_await();
  else if constexpr (has_non_member_operator_co_await_v<Awaitable&&>)
    return operator co_await(static_cast<Awaitable&&>(awaitable));
  else
    return static_cast<Awaitable&&>(awaitable);
}
```

Awaiting the Awaiter

So, assuming we have encapsulated the logic for turning the `<expr>` result into an **Awaiter** object into the above functions then the semantics of `co_await <expr>` can be translated (roughly) as follows:

```
{
  auto&& value = <expr>;
  auto&& awaitable = get_awaitable(promise, static_cast<decltype(value)>(value));
  auto&& awaiter = get_awaiter(static_cast<decltype(awaitable)>(awaitable));
  if (!awaiter.await_ready())
  {
    using handle_t = std::experimental::coroutine_handle<P>;

    using await_suspend_result_t =
      decltype(awaiter.await_suspend(handle_t::from_promise(p)));

    <suspend-coroutine>

    if constexpr (std::is_void_v<await_suspend_result_t>)
    {
      awaiter.await_suspend(handle_t::from_promise(p));
      <return-to-caller-or-resumer>
    }
    else
    {
      static_assert(
         std::is_same_v<await_suspend_result_t, bool>,
         "await_suspend() must return 'void' or 'bool'.");

      if (awaiter.await_suspend(handle_t::from_promise(p)))
      {
        <return-to-caller-or-resumer>
      }
    }

    <resume-point>
  }

  return awaiter.await_resume();
}
```

The `void` -returning version of `await_suspend()` unconditionally transfers execution back to the caller/resumer of the coroutine when the call to `await_suspend()` returns, whereas the `bool` -returning version allows the awaiter object to conditionally resume the coroutine immediately without returning to the caller/resumer.

The `bool` -returning version of `await_suspend()` can be useful in cases where the awaiter might start an async operation that can sometimes complete synchronously. In the cases where it completes synchronously, the `await_suspend()` method can return `false` to indicate that the coroutine should be immediately resumed and continue execution.

At the `<suspend-coroutine>` point the compiler generates some code to save the current state of the coroutine and prepare it for resumption. This includes storing the location of the `<resume-point>` as well as spilling any values currently held in registers into the coroutine frame memory.

The current coroutine is considered suspended after the `<suspend-coroutine>` operation completes. The first point at which you can observe the suspended coroutine is inside the call to `await_suspend()` . Once the coroutine is suspended it is then able to be resumed or destroyed.

It is the responsibility of the `await_suspend()` method to schedule the coroutine for resumption (or destruction) at some point in the future once the operation has completed. Note that returning `false` from `await_suspend()` counts as scheduling the coroutine for immediate resumption on the current thread.

The purpose of the `await_ready()` method is to allow you to avoid the cost of the `<suspend-coroutine>` operation in cases where it is known that the operation will complete synchronously without needing to suspend.

At the `<return-to-caller-or-resumer>` point execution is transferred back to the caller or resumer, popping the local stack frame but keeping the coroutine frame alive.

When (or if) the suspended coroutine is eventually resumed then the execution resumes at the `<resume-point>` . ie. immediately before the `await_resume()` method is called to obtain the result of the operation.

The return-value of the `await_resume()` method call becomes the result of the `co_await` expression. The `await_resume()` method can also throw an exception in which case the exception propagates out of the `co_await` expression.

Note that if an exception propagates out of the `await_suspend()` call then the coroutine is automatically resumed and the exception propagates out of the `co_await` expression without calling `await_resume()` .

Coroutine Handles

You may have noticed the use of the `coroutine_handle<P>` type that is passed to the `await_suspend()` call of a `co_await` expression.

This type represents a non-owning handle to the coroutine frame and can be used to resume execution of the coroutine or to destroy the coroutine frame. It can also be used to get access to the coroutine's promise object.

The `coroutine_handle` type has the following (abbreviated) interface:

```
namespace std::experimental
{
  template<typename Promise>
  struct coroutine_handle;

  template<>
  struct coroutine_handle<void>
  {
    bool done() const;

    void resume();
    void destroy();

    void* address() const;
    static coroutine_handle from_address(void* address);
  };

  template<typename Promise>
  struct coroutine_handle : coroutine_handle<void>
  {
    Promise& promise() const;
    static coroutine_handle from_promise(Promise& promise);

    static coroutine_handle from_address(void* address);
  };
}
```

When implementing **Awaitable** types, they key method you'll be using on `coroutine_handle` will be `.resume()`, which should be called when the operation has completed and you want to resume execution of the awaiting coroutine. Calling `.resume()` on a `coroutine_handle` reactivates a suspended coroutine at the `<resume-point>`. The call to `.resume()` will return when the coroutine next hits a `<return-to-caller-or-resumer>` point.

The `.destroy()` method destroys the coroutine frame, calling the destructors of any in-scope variables and freeing memory used by the coroutine frame. You should generally not need to (and indeed should really avoid) calling `.destroy()` unless you are a library writer implementing the coroutine promise type. Normally, coroutine frames will be owned by some kind of RAII type returned from the call to the coroutine. So calling `.destroy()` without cooperation with the RAII object could lead to a double-destruction bug.

The `.promise()` method returns a reference to the coroutine's promise object. However, like `.destroy()`, it is generally only useful if you are authoring coroutine promise types. You should consider the coroutine's promise object as an internal implementation detail of the coroutine. For most **Normally Awaitable** types you should use `coroutine_handle<void>` as the parameter type to the `await_suspend()` method instead of `coroutine_handle<Promise>`.

The `coroutine_handle<P>::from_promise(P& promise)` function allows reconstructing the coroutine handle from a reference to the coroutine's promise object. Note that you must ensure that the type, `P`, exactly matches the concrete promise type used for the coroutine frame; attempting to construct a `coroutine_handle<Base>` when the concrete promise type is `Derived` can lead to undefined behaviour.

The `.address()` / `from_address()` functions allow converting a coroutine handle to/from a `void*` pointer. This is primarily intended to allow passing as a 'context' parameter into existing C-style APIs, so you might find it useful in implementing **Awaitable** types in some circumstances. However, in most cases I've found it necessary to pass additional information through to callbacks in this 'context' parameter so I generally end up storing the `coroutine_handle` in a struct and passing a pointer to the struct in the 'context' parameter rather than using the `.address()` return-value.

Synchronisation-free async code

One of the powerful design-features of the `co_await` operator is the ability to execute code after the coroutine has been suspended but before execution is returned to the caller/resumer.

This allows an Awaiter object to initiate an async operation after the coroutine is already suspended, passing the `coroutine_handle` of the suspended coroutine to the operation which it can safely resume when the operation completes (potentially on another thread) without any additional synchronisation required.

For example, by starting an async-read operation inside `await_suspend()` when the coroutine is already suspended means that we can just resume the coroutine when the operation completes without needing any thread-synchronisation to coordinate the thread that started the operation and the thread that completed the operation.

```
Time     Thread 1                          Thread 2
  |      --------                          --------
  |      ....                              Call OS - Wait for I/O event
  |      Call await_ready()                   |
  |      <supend-point>                       |
  |      Call await_suspend(handle)           |
  |        Store handle in operation          |
  V        Start AsyncFileRead ---+           V
                                  +----->  <AsyncFileRead Completion Event>
                                           Load coroutine_handle from operation
                                           Call handle.resume()
                                             <resume-point>
                                             Call to await_resume()
                                             execution continues....
           Call to AsyncFileRead returns
         Call to await_suspend() returns
         <return-to-caller/resumer>
```

One thing to be *very* careful of when taking advantage of this approach is that as soon as you have started the operation which publishes the coroutine handle to other threads then another thread may resume the coroutine on another thread before `await_suspend()` returns and may continue executing concurrently with the rest of the `await_suspend()` method.

The first thing the coroutine will do when it resumes is call `await_resume()` to get the result and then often it will immediately destruct the **Awaiter** object (ie. the `this` pointer of the `await_suspend()` call). The coroutine could then potentially run to completion, destructing the coroutine and promise object, all before `await_suspend()` returns.

So within the `await_suspend()` method, once it's possible for the coroutine to be resumed concurrently on another thread, you need to make sure that you avoid accessing `this` or the coroutine's `.promise()` object because both could already be destroyed. In general, the only things that are safe to access after the operation is started and the coroutine is scheduled for resumption are local variables within `await_suspend()`.

Comparison to Stackful Coroutines

I want to take a quick detour to compare this ability of the Coroutines TS stackless coroutines to execute logic after the coroutine is suspended with some existing common stackful coroutine facilities such as Win32 fibers or boost::context.

With many of the stackful coroutine frameworks, the suspend operation of a coroutine is combined with the resumption of another coroutine into a 'context-switch' operation. With this 'context-switch' operation there is typically no opportunity to execute logic after suspending the current coroutine but before transferring execution to another coroutine.

This means that if we want to implement a similar async-file-read operation on top of stackful coroutines then we have to start the operation *before* suspending the coroutine. It is therefore possible that the operation could complete on another thread before the coroutine is suspended and is eligible for resumption. This potential race between the operation completing on another thread and the coroutine suspending requires some kind of thread synchronisation to arbitrate and decide on the winner.

There are probably ways around this by using a trampoline context that can start the operation on behalf of the initiating context after the initiating context has been suspended. However this would require extra infrastructure and an extra context-switch to make it work and it's possible that the overhead this introduces would be greater than the cost of the synchronisation it's trying to avoid.

Avoiding memory allocations

Async operations often need to store some per-operation state that keeps track of the progress of the operation. This state typically needs to last for the duration of the operation and should only be freed once the operation has completed.

For example, calling async Win32 I/O functions requires you to allocate and pass a pointer to an `OVERLAPPED` structure. The caller is responsible for ensuring this pointer remains valid until the operation completes.

With traditional callback-based APIs this state would typically need to be allocated on the heap to ensure it has the appropriate lifetime. If you were performing many operations, you may need to allocate and free this state for each operation. If performance is an issue then a custom allocator may be used that allocates these state objects from a pool.

However, when we are using coroutines we can avoid the need to heap-allocate storage for the operation state by taking advantage of the fact that local variables within the coroutine frame will be kept alive while the coroutine is suspended.

By placing the per-operation state in the **Awaiter** object we can effectively "borrow" memory from the coroutine frame for storing the per-operation state for the duration of the `co_await` expression. Once the operation completes, the coroutine is resumed and the **Awaiter** object is destroyed, freeing that memory in the coroutine frame for use by other local variables.

Ultimately, the coroutine frame may still be allocated on the heap. However, once allocated, a coroutine frame can be used to execute many asynchronous operations with only that single heap allocation.

If you think about it, the coroutine frame acts as a kind of really high-performance arena memory allocator. The compiler figures out at compile time the total arena size it needs for all local variables and is then able to allocate this memory out to local variables as required with zero overhead! Try beating that with a custom allocator ;)

An example: Implementing a simple thread-synchronisation primitive

Now that we've covered a lot of the mechanics of the `co_await` operator, I want to show how to put some of this knowledge into practice by implementing a basic awaitable synchronisation primitive: An asynchronous manual-reset event.

The basic requirements of this event is that it needs to be **Awaitable** by multiple concurrently executing coroutines and when awaited needs to suspend the awaiting coroutine until some thread calls the `.set()` method, at which point any awaiting coroutines are resumed. If some thread has already called `.set()` then the coroutine should continue without suspending.

Ideally we'd also like to make it `noexcept`, require no heap allocations and have a lock-free implementation.

Edit 2017/11/23: Added example usage for `async_manual_reset_event`

Example usage should look something like this:

```
T value;
async_manual_reset_event event;

// A single call to produce a value
void producer()
{
  value = some_long_running_computation();

  // Publish the value by setting the event.
  event.set();
}

// Supports multiple concurrent consumers
task<> consumer()
{
  // Wait until the event is signalled by call to event.set()
  // in the producer() function.
  co_await event;

  // Now it's safe to consume 'value'
  // This is guaranteed to 'happen after' assignment to 'value'
  std::cout << value << std::endl;
}
```

Let's first think about the possible states this event can be in: 'not set' and 'set'.

When it's in the 'not set' state there is a (possibly empty) list of waiting coroutines that are waiting for it to become 'set'.

When it's in the 'set' state there won't be any waiting coroutines as coroutines that `co_await` the event in this state can continue without suspending.

This state can actually be represented in a single `std::atomic<void*>` .

- Reserve a special pointer value for the 'set' state. In this case we'll use the `this` pointer of the event since we know that can't be the same address as any of the list items.
- Otherwise the event is in the 'not set' state and the value is a pointer to the head of a singly linked-list of awaiting coroutine structures.

We can avoid extra calls to allocate nodes for the linked-list on the heap by storing the nodes within an 'awaiter' object that is placed within the coroutine frame.

So let's start with a class interface that looks something like this:

```
class async_manual_reset_event
{
public:

  async_manual_reset_event(bool initiallySet = false) noexcept;

  // No copying/moving
  async_manual_reset_event(const async_manual_reset_event&) = delete;
  async_manual_reset_event(async_manual_reset_event&&) = delete;
  async_manual_reset_event& operator=(const async_manual_reset_event&) = delete;
  async_manual_reset_event& operator=(async_manual_reset_event&&) = delete;

  bool is_set() const noexcept;

  struct awaiter;
  awaiter operator co_await() const noexcept;

  void set() noexcept;
  void reset() noexcept;

private:

  friend struct awaiter;

  // - 'this' => set state
  // - otherwise => not set, head of linked list of awaiter*.
  mutable std::atomic<void*> m_state;

};
```

Here we have a fairly straight-forward and simple interface. The main thing to note at this point is that it has an `operator co_await()` method that returns an, as yet, undefined type, `awaiter` .

Let's define the `awaiter` type now.

Defining the Awaiter

Firstly, it needs to know which `async_manual_reset_event` object it is going to be awaiting, so it will need a reference to the event and a constructor to initialise it.

It also needs to act as a node in a linked-list of `awaiter` values so it will need to hold a pointer to the next `awaiter` object in the list.

It also needs to store the `coroutine_handle` of the awaiting coroutine that is executing the `co_await` expression so that the event can resume the coroutine when it becomes 'set'. We don't care what the promise type of the coroutine is so we'll just use a `coroutine_handle<>` (which is short-hand for `coroutine_handle<void>`).

Finally, it needs to implement the **Awaiter** interface, so it needs the three special methods: `await_ready` , `await_suspend` and `await_resume` . We don't need to return a value from the `co_await` expression so `await_resume` can return `void` .

Once we put all of that together, the basic class interface for `awaiter` looks like this:

```
struct async_manual_reset_event::awaiter
{
  awaiter(const async_manual_reset_event& event) noexcept
  : m_event(event)
  {}

  bool await_ready() const noexcept;
  bool await_suspend(std::experimental::coroutine_handle<> awaitingCoroutine) noexcept;
  void await_resume() noexcept {}

private:

  const async_manual_reset_event& m_event;
  std::experimental::coroutine_handle<> m_awaitingCoroutine;
  awaiter* m_next;
};
```

Now, when we `co_await` an event, we don't want the awaiting coroutine to suspend if the event is already set. So we can define `await_ready()` to return `true` if the event is already set.

```
bool async_manual_reset_event::awaiter::await_ready() const noexcept
{
  return m_event.is_set();
}
```

Next, let's look at the `await_suspend()` method. This is usually where most of the magic happens in an awaitable type.

First it will need to stash the coroutine handle of the awaiting coroutine into the `m_awaitingCoroutine` member so that the event can later call `.resume()` on it.

Then once we've done that we need to try and atomically enqueue the awaiter onto the linked list of waiters. If we successfully enqueue it then we return `true` to indicate that we don't want to resume the coroutine immediately, otherwise if we find that the event has concurrently been changed to the 'set' state then we return `false` to indicate that the coroutine should be resumed immediately.

```
bool async_manual_reset_event::awaiter::await_suspend(
  std::experimental::coroutine_handle<> awaitingCoroutine) noexcept
{
  // Special m_state value that indicates the event is in the 'set' state.
  const void* const setState = &m_event;

  // Remember the handle of the awaiting coroutine.
  m_awaitingCoroutine = awaitingCoroutine;

  // Try to atomically push this awaiter onto the front of the list.
  void* oldValue = m_event.m_state.load(std::memory_order_acquire);
  do
  {
    // Resume immediately if already in 'set' state.
    if (oldValue == setState) return false;

    // Update linked list to point at current head.
    m_next = static_cast<awaiter*>(oldValue);

    // Finally, try to swap the old list head, inserting this awaiter
    // as the new list head.
  } while (!m_event.m_state.compare_exchange_weak(
             oldValue,
             this,
             std::memory_order_release,
             std::memory_order_acquire));

  // Successfully enqueued. Remain suspended.
  return true;
}
```

Note that we use ‘acquire’ memory order when loading the old state so that if we read the special ‘set’ value then we have visibility of writes that occurred prior to the call to ‘set()’.

We require ‘release’ sematics if the compare-exchange succeeds so that a subsequent call to ‘set()’ will see our writes to m_awaitingCoroutine and prior writes to the coroutine state.

Filling out the rest of the event class

Now that we have defined the `awaiter` type, let’s go back and look at the implementation of the `async_manual_reset_event` methods.

First, the constructor. It needs to initialise to either the ‘not set’ state with the empty list of waiters (ie. `nullptr`) or initialise to the ‘set’ state (ie. `this`).

```
async_manual_reset_event::async_manual_reset_event(
  bool initiallySet) noexcept
: m_state(initiallySet ? this : nullptr)
{}
```

Next, the `is_set()` method is pretty straight-forward - it’s ‘set’ if it has the special value `this` :

```
bool async_manual_reset_event::is_set() const noexcept
{
  return m_state.load(std::memory_order_acquire) == this;
}
```

Next, the `reset()` method. If it's in the 'set' state we want to transition back to the empty-list 'not set' state, otherwise leave it as it is.

```
void async_manual_reset_event::reset() noexcept
{
  void* oldValue = this;
  m_state.compare_exchange_strong(oldValue, nullptr, std::memory_order_acquire);
}
```

With the `set()` method, we want to transition to the 'set' state by exchanging the current state with the special 'set' value, `this`, and then examine what the old value was. If there were any waiting coroutines then we want to resume each of them sequentially in turn before returning.

```
void async_manual_reset_event::set() noexcept
{
  // Needs to be 'release' so that subsequent 'co_await' has
  // visibility of our prior writes.
  // Needs to be 'acquire' so that we have visibility of prior
  // writes by awaiting coroutines.
  void* oldValue = m_state.exchange(this, std::memory_order_acq_rel);
  if (oldValue != this)
  {
    // Wasn't already in 'set' state.
    // Treat old value as head of a linked-list of waiters
    // which we have now acquired and need to resume.
    auto* waiters = static_cast<awaiter*>(oldValue);
    while (waiters != nullptr)
    {
      // Read m_next before resuming the coroutine as resuming
      // the coroutine will likely destroy the awaiter object.
      auto* next = waiters->m_next;
      waiters->m_awaitingCoroutine.resume();
      waiters = next;
    }
  }
}
```

Finally, we need to implement the `operator co_await()` method. This just needs to construct an `awaiter` object.

```
async_manual_reset_event::awaiter
async_manual_reset_event::operator co_await() const noexcept
{
  return awaiter{ *this };
}
```

And there we have it. An awaitable asynchronous manual-reset event that has a lock-free, memory-allocation-free, `noexcept` implementation.

If you want to have a play with the code or check out what it compiles down to under MSVC and Clang have a look at the source on godbolt.

You can also find an implementation of this class available in the cppcoro library, along with a number of other useful awaitable types such as `async_mutex` and `async_auto_reset_event` .

Closing Off

This post has looked at how the `operator co_await` is implemented and defined in terms of the **Awaitable** and **Awaiter** concepts.

It has also walked through how to implement an awaitable async thread-synchronisation primitive that takes advantage of the fact that awaiter objects are allocated on the coroutine frame to avoid additional heap allocations.

I hope this post has helped to demystify the new `co_await` operator for you.

In the next post I'll explore the **Promise** concept and how a coroutine-type author can customise the behaviour of their coroutine.

Thanks

I want to call out special thanks to Gor Nishanov for patiently and enthusiastically answering my many questions on coroutines over the last couple of years.

And also to Eric Niebler for reviewing and providing feedback on an early draft of this post.

C++ Coroutines: Understanding the promise type

lewissbaker.github.io/2018/09/05/understanding-the-promise-type

Asymmetric Transfer 5 September 2018

Sep 5, 2018

This post is the third in the series on the C++ Coroutines TS (N4736).

The previous articles in this series cover:

In this post I look at the mechanics of how the compiler translates coroutine code that you write into compiled code and how you can customise the behaviour of a coroutine by defining your own **Promise** type.

Coroutine Concepts

The Coroutines TS adds three new keywords: `co_await` , `co_yield` and `co_return` . Whenever you use one of these coroutine keywords in the body of a function this triggers the compiler to compile this function as a coroutine rather than as a normal function.

The compiler applies some fairly mechanical transformations to the code that you write to turn it into a state-machine that allows it to suspend execution at particular points within the function and then later resume execution.

In the previous post I described the first of two new interfaces that the Coroutines TS introduces: The **Awaitable** interface. The second interface that the TS introduces that is important to this code transformation is the **Promise** interface.

The **Promise** interface specifies methods for customising the behaviour of the coroutine itself. The library-writer is able to customise what happens when the coroutine is called, what happens when the coroutine returns (either by normal means or via an unhandled exception) and customise the behaviour of any `co_await` or `co_yield` expression within the coroutine.

Promise objects

The **Promise** object defines and controls the behaviour of the coroutine itself by implementing methods that are called at specific points during execution of the coroutine.

> Before we go on, I want you to try and rid yourself of any preconceived notions of what a "promise" is. While, in some use-cases, the coroutine promise object does indeed act in a similar role to the `std::promise` part of a `std::future` pair, for other use-cases the analogy is somewhat stretched. It may be easier to think about the coroutine's promise object as being a "coroutine state controller" object that controls the behaviour of the coroutine and can be used to track its state.

An instance of the promise object is constructed within the coroutine frame for each invocation of a coroutine function.

The compiler generates calls to certain methods on the promise object at key points during execution of the coroutine.

In the following examples, assume that the promise object created in the coroutine frame for a particular invocation of the coroutine is `promise` .

When you write a coroutine function that has a body, `<body-statements>` , which contains one of the coroutine keywords (`co_return` , `co_await` , `co_yield`) then the body of the coroutine is transformed to something (roughly) like the following:

```
{
  co_await promise.initial_suspend();
  try
  {
    <body-statements>
  }
  catch (...)
  {
    promise.unhandled_exception();
  }
FinalSuspend:
  co_await promise.final_suspend();
}
```

When a coroutine function is called there are a number of steps that are performed prior to executing the code in the source of the coroutine body that are a little different to regular functions.

Here is a summary of the steps (I'll go into more detail on each of the steps below).

1. Allocate a coroutine frame using `operator new` (optional).
2. Copy any function parameters to the coroutine frame.
3. Call the constructor for the promise object of type, `P` .
4. Call the `promise.get_return_object()` method to obtain the result to return to the caller when the coroutine first suspends. Save the result as a local variable.
5. Call the `promise.initial_suspend()` method and `co_await` the result.
6. When the `co_await promise.initial_suspend()` expression resumes (either immediately or asynchronously), then the coroutine starts executing the coroutine body statements that you wrote.

Some additional steps are executed when execution reaches a `co_return` statement:

1. Call `promise.return_void()` or `promise.return_value(<expr>)`
2. Destroy all variables with automatic storage duration in reverse order they were created.
3. Call `promise.final_suspend()` and `co_await` the result.

If instead, execution leaves `<body-statements>` due to an unhandled exception then:

1. Catch the exception and call `promise.unhandled_exception()` from within the catch-block.
2. Call `promise.final_suspend()` and `co_await` the result.

Once execution propagates outside of the coroutine body then the coroutine frame is destroyed. Destroying the coroutine frame involves a number of steps:

1. Call the destructor of the promise object.
2. Call the destructors of the function parameter copies.
3. Call `operator delete` to free the memory used by the coroutine frame (optional)
4. Transfer execution back to the caller/resumer.

When execution first reaches a `<return-to-caller-or-resumer>` point inside a `co_await` expression, or if the coroutine runs to completion without hitting a `<return-to-caller-or-resumer>` point, then the coroutine is either suspended or destroyed and the return-object previously returned from the call to `promise.get_return_object()` is then returned to the caller of the coroutine.

Allocating a coroutine frame

First, the compiler generates a call to `operator new` to allocate memory for the coroutine frame.

If the promise type, `P` , defines a custom `operator new` method then that is called, otherwise the global `operator new` is called.

There are a few important things to note here:

The size passed to `operator new` is not `sizeof(P)` but is rather the size of the entire coroutine frame and is determined automatically by the compiler based on the number and sizes of parameters, size of the promise object, number and sizes of local variables and other compiler-specific storage needed for management of coroutine state.

The compiler is free to elide the call to `operator new` as an optimisation if:

- it is able to determine that the lifetime of the coroutine frame is strictly nested within the lifetime of the caller; and
- the compiler can see the size of coroutine frame required at the call-site.

In these cases, the compiler can allocate storage for the coroutine frame in the caller's activation frame (either in the stack-frame or coroutine-frame part).

The Coroutines TS does not yet specify any situations in which the allocation elision is guaranteed, so you still need to write code as if the allocation of the coroutine frame may fail with `std::bad_alloc` . This also means that you usually shouldn't declare a coroutine function as `noexcept` unless you are ok with `std::terminate()` being called if the coroutine fails to allocate memory for the coroutine frame.

There is a fallback, however, that can be used in lieu of exceptions for handling failure to allocate the coroutine frame. This can be necessary when operating in environments where exceptions are not allowed, such as embedded environments or high-performance environments where the overhead of exceptions is not tolerated.

If the promise type provides a static `P::get_return_object_on_allocation_failure()` member function then the compiler will generate a call to the `operator new(size_t, nothrow_t)` overload instead. If that call returns `nullptr` then the coroutine will immediately call `P::get_return_object_on_allocation_failure()` and return the result to the caller of the coroutine instead of throwing an exception.

Customising coroutine frame memory allocation

Your promise type can define an overload of `operator new()` that will be called instead of global-scope `operator new` if the compiler needs to allocate memory for a coroutine frame that uses your promise type.

For example:

```
struct my_promise_type
{
  void* operator new(std::size_t size)
  {
    void* ptr = my_custom_allocate(size);
    if (!ptr) throw std::bad_alloc{};
    return ptr;
  }

  void operator delete(void* ptr, std::size_t size)
  {
    my_custom_free(ptr, size);
  }

  ...
};
```

“But what about custom allocators?”, I hear you asking.

You can also provide an overload of `P::operator new()` that takes additional arguments which will be called with lvalue references to the coroutine function parameters if a suitable overload can be found. This can be used to hook up `operator new` to call an `allocate()` method on an allocator that was passed as an argument to the coroutine function.

You will need to do some extra work to make a copy of the allocator inside the allocated memory so you can reference it in the corresponding call to `operator delete` since the parameters are not passed to the corresponding `operator delete` call. This is because the parameters are stored in the coroutine-frame and so they will have already been destructed by the time that `operator delete` is called.

For example, you can implement `operator new` so that it allocates extra space after the coroutine frame and use that space to stash a copy of the allocator that can be used to free the coroutine frame memory.

For example:

```
template<typename ALLOCATOR>
struct my_promise_type
{
  template<typename... ARGS>
  void* operator new(std::size_t sz, std::allocator_arg_t, ALLOCATOR& allocator, ARGS&... args)
  {
    // Round up sz to next multiple of ALLOCATOR alignment
    std::size_t allocatorOffset =
      (sz + alignof(ALLOCATOR) - 1u) & ~(alignof(ALLOCATOR) - 1u);

    // Call onto allocator to allocate space for coroutine frame.
    void* ptr = allocator.allocate(allocatorOffset + sizeof(ALLOCATOR));

    // Take a copy of the allocator (assuming noexcept copy constructor here)
    new (((char*)ptr) + allocatorOffset) ALLOCATOR(allocator);

    return ptr;
  }

  void operator delete(void* ptr, std::size_t sz)
  {
    std::size_t allocatorOffset =
      (sz + alignof(ALLOCATOR) - 1u) & ~(alignof(ALLOCATOR) - 1u);

    ALLOCATOR& allocator = *reinterpret_cast<ALLOCATOR*>(
      ((char*)ptr) + allocatorOffset);

    // Move allocator to local variable first so it isn't freeing its
    // own memory from underneath itself.
    // Assuming allocator move-constructor is noexcept here.
    ALLOCATOR allocatorCopy = std::move(allocator);

    // But don't forget to destruct allocator object in coroutine frame
    allocator.~ALLOCATOR();

    // Finally, free the memory using the allocator.
    allocatorCopy.deallocate(ptr, allocatorOffset + sizeof(ALLOCATOR));
  }
}
```

To hook up the custom `my_promise_type` to be used for coroutines that pass `std::allocator_arg` as the first parameter, you need to specialise the `coroutine_traits` class (see section on `coroutine_traits` below for more details).

For example:

```
namespace std::experimental
{
  template<typename ALLOCATOR, typename... ARGS>
  struct coroutine_traits<my_return_type, std::allocator_arg_t, ALLOCATOR, ARGS...>
  {
    using promise_type = my_promise_type<ALLOCATOR>;
  };
}
```

Note that even if you customise the memory allocation strategy for a coroutine, **the compiler is still allowed to elide the call to your memory allocator**.

Copying parameters to the coroutine frame

The coroutine needs to copy any parameters passed to the coroutine function by the original caller into the coroutine frame so that they remain valid after the coroutine is suspended.

If parameters are passed to the coroutine by value, then those parameters are copied to the coroutine frame by calling the type's move-constructor.

If parameters are passed to the coroutine by reference (either lvalue or rvalue), then only the references are copied into the coroutine frame, not the values they point to.

Note that for types with trivial destructors, the compiler is free to elide the copy of the parameter if the parameter is never referenced after a reachable `<return-to-caller-or-resumer>` point in the coroutine.

There are many gotchas involved when passing parameters by reference into coroutines as you cannot necessarily rely on the reference remaining valid for the lifetime of the coroutine. Many common techniques used with normal functions, such as perfect-forwarding and universal-references, can result in code that has undefined behaviour if used with coroutines. Toby Allsopp has written a great article on this topic if you want more details.

If any of the parameter copy/move constructors throws an exception then any parameters already constructed are destructed, the coroutine frame is freed and the exception propagates back out to the caller.

Constructing the promise object

Once all of the parameters have been copied into the coroutine frame, the coroutine then constructs the promise object.

The reason the parameters are copied prior to the promise object being constructed is to allow the promise object to be given access to the post-copied parameters in its constructor.

First, the compiler checks to see if there is an overload of the promise constructor that can accept lvalue references to each of the copied parameters. If the compiler finds such an overload then the compiler generates a call to that constructor overload. If it does not find such an overload then the compiler falls back to generating a call to the promise type's default constructor.

Note that the ability for the promise constructor to "peek" at the parameters was a relatively recent change to the Coroutines TS, being adopted in N4723 at the Jacksonville 2018 meeting. See P0914R1 for the proposal. Thus it may not be supported by some older versions of Clang or MSVC.

If the promise constructor throws an exception then the parameter copies are destructed and the coroutine frame freed during stack unwinding before the exception propagates out to the caller.

Obtaining the return object

The first thing a coroutine does with the promise object is obtain the `return-object` by calling `promise.get_return_object()` .

The `return-object` is the value that is returned to the caller of the coroutine function when the coroutine first suspends or after it runs to completion and execution returns to the caller.

You can think of the control flow going something (very roughly) like this:

```
// Pretend there's a compiler-generated structure called 'coroutine_frame'
// that holds all of the state needed for the coroutine. It's constructor
// takes a copy of parameters and default-constructs a promise object.
struct coroutine_frame { ... };

T some_coroutine(P param)
{
  auto* f = new coroutine_frame(std::forward<P>(param));

  auto returnObject = f->promise.get_return_object();

  // Start execution of the coroutine body by resuming it.
  // This call will return when the coroutine gets to the first
  // suspend-point or when the coroutine runs to completion.
  coroutine_handle<decltype(f->promise)>::from_promise(f->promise).resume();

  // Then the return object is returned to the caller.
  return returnObject;
}
```

Note that we need to obtain the return-object before starting the coroutine body since the coroutine frame (and thus the promise object) may be destroyed prior to the call to `coroutine_handle::resume()` returning, either on this thread or possibly on another thread, and so it would be unsafe to call `get_return_object()` after starting execution of the coroutine body.

The initial-suspend point

The next thing the coroutine executes once the coroutine frame has been initialised and the return object has been obtained is execute the statement `co_await promise.initial_suspend();` .

This allows the author of the `promise_type` to control whether the coroutine should suspend before executing the coroutine body that appears in the source code or start executing the coroutine body immediately.

If the coroutine suspends at the initial suspend point then it can be later resumed or destroyed at a time of your choosing by calling `resume()` or `destroy()` on the coroutine's `coroutine_handle` .

The result of the `co_await promise.initial_suspend()` expression is discarded so implementations should generally return `void` from the `await_resume()` method of the awaiter.

It is important to note that this statement exists outside of the `try` / `catch` block that guards the rest of the coroutine (scroll back up to the definition of the coroutine body if you've forgotten what it looks like). This means that any exception thrown from the `co_await promise.initial_suspend()` evaluation prior to hitting its `<return-to-caller-or-resumer>` will be thrown back to the caller of the coroutine after destroying the coroutine frame and the return object.

Be aware of this if your `return-object` has RAII semantics that destroy the coroutine frame on destruction. If this is the case then you want to make sure that `co_await promise.initial_suspend()` is `noexcept` to avoid double-free of the coroutine frame.

> Note that there is a proposal to tweak the semantics so that either all or part of the `co_await promise.initial_suspend()` expression lies inside try/catch block of the coroutine-body so the exact semantics here are likely to change before coroutines are finalised.

For many types of coroutine, the `initial_suspend()` method either returns `std::experimental::suspend_always` (if the operation is lazily started) or `std::experimental::suspend_never` (if the operation is eagerly started) which are both `noexcept` awaitables so this is usually not an issue.

Returning to the caller

When the coroutine function reaches its first `<return-to-caller-or-resumer>` point (or if no such point is reached then when execution of the coroutine runs to completion) then the `return-object` returned from the `get_return_object()` call is returned to the caller of the coroutine.

Note that the type of the `return-object` doesn't need to be the same type as the return-type of the coroutine function. An implicit conversion from the `return-object` to the return-type of the coroutine is performed if necessary.

> Note that Clang's implementation of coroutines (as of 5.0) defers executing this conversion until the return-object is returned from the coroutine call, whereas MSVC's implementation as of 2017 Update 3 performs the conversion immediately after calling `get_return_object()`. Although the Coroutines TS is not explicit on the intended behaviour, I believe MSVC has plans to change their implementation to behave more like Clang's as this enables some interesting use cases.

Returning from the coroutine using `co_return`

When the coroutine reaches a `co_return` statement, it is translated into either a call to `promise.return_void()` or `promise.return_value(<expr>)` followed by a `goto FinalSuspend;`.

The rules for the translation are as follows:

- `co_return;`
  `->` `promise.return_void();`

- `co_return <expr>;`
  -> `<expr>; promise.return_void();` if `<expr>` has type `void`
  -> `promise.return_value(<expr>);` if `<expr>` does not have type `void`

The subsequent `goto FinalSuspend;` causes all local variables with automatic storage duration to be destructed in reverse order of construction before then evaluating `co_await promise.final_suspend();` .

Note that if execution runs off the end of a coroutine without a `co_return` statement then this is equivalent to having a `co_return;` at the end of the function body. In this case, if the `promise_type` does not have a `return_void()` method then the behaviour is undefined.

If either the evaluation of `<expr>` or the call to `promise.return_void()` or `promise.return_value()` throws an exception then the exception still propagates to `promise.unhandled_exception()` (see below).

Handling exceptions that propagate out of the coroutine body

If an exception propagates out of the coroutine body then the exception is caught and the `promise.unhandled_exception()` method is called inside the `catch` block.

Implementations of this method typically call `std::current_exception()` to capture a copy of the exception to store it away to be later rethrown in a different context.

Alternatively, the implementation could immediately rethrow the exception by executing a `throw;` statement. For example see folly::Optional However, doing so will (likely - see below) cause the the coroutine frame to be immediately destroyed and for the exception to propagate out to the caller/resumer. This could cause problems for some abstractions that assume/require the call to `coroutine_handle::resume()` to be `noexcept` , so you should generally only use this approach when you have full control over who/what calls `resume()` .

Note that the current Coroutines TS wording is a little unclear on the intended behaviour if the call to `unhandled_exception()` rethrows the exception (or for that matter if any of the logic outside of the try-block throws an exception).

My current interpretation of the wording is that if control exits the coroutine-body, either via exception propagating out of `co_await promise.initial_suspend()` , `promise.unhandled_exception()` or `co_await promise.final_suspend()` or by the coroutine running to completion by `co_await p.final_suspend()` completing synchronously then the coroutine frame is automatically destroyed before execution returns to the caller/resumer. However, this interpretation has its own issues.

A future version of the Coroutines specification will hopefully clarify the situation. However, until then I'd stay away from throwing exceptions out of `initial_suspend()` , `final_suspend()` or `unhandled_exception()` . Stay tuned!

The final-suspend point

Once execution exits the user-defined part of the coroutine body and the result has been captured via a call to `return_void()` , `return_value()` or `unhandled_exception()` and any local variables have been destructed, the coroutine has an opportunity to execute some additional logic before execution is returned back to the caller/resumer.

The coroutine executes the `co_await promise.final_suspend();` statement.

This allows the coroutine to execute some logic, such as publishing a result, signalling completion or resuming a continuation. It also allows the coroutine to optionally suspend immediately before execution of the coroutine runs to completion and the coroutine frame is destroyed.

Note that it is undefined behaviour to `resume()` a coroutine that is suspended at the `final_suspend` point. The only thing you can do with a coroutine suspended here is `destroy()` it.

The rationale for this limitation, according to Gor Nishanov, is that this provides several optimisation opportunities for the compiler due to the reduction in the number of suspend states that need to be represented by the coroutine and a potential reduction in the number of branches required.

Note that while it is allowed to have a coroutine not suspend at the `final_suspend` point, **it is recommended that you structure your coroutines so that they do suspend at `final_suspend`** where possible. This is because this forces you to call `.destroy()` on the coroutine from outside of the coroutine (typically from some RAII object destructor) and this makes it much easier for the compiler to determine when the scope of the lifetime of the coroutine-frame is nested inside the caller. This in turn makes it much more likely that the compiler can elide the memory allocation of the coroutine frame.

How the compiler chooses the promise type

So lets look now at how the compiler determines what type of promise object to use for a given coroutine.

The type of the promise object is determined from the signature of the coroutine by using the `std::experimental::coroutine_traits` class.

If you have a coroutine function with signature:

```
task<float> foo(std::string x, bool flag);
```

Then the compiler will deduce the type of the coroutine's promise by passing the return-type and parameter types as template arguments to `coroutine_traits` .

```
typename coroutine_traits<task<float>, std::string, bool>::promise_type;
```

If the function is a non-static member function then the class type is passed as the second template parameter to `coroutine_traits` . Note that if your method is overloaded for rvalue-references then the second template parameter will be an rvalue reference.

For example, if you have the following methods:

```
task<void> my_class::method1(int x) const;
task<foo> my_class::method2() &&;
```

Then the compiler will use the following promise types:

```
// method1 promise type
typename coroutine_traits<task<void>, const my_class&, int>::promise_type;

// method2 promise type
typename coroutine_traits<task<foo>, my_class&&>::promise_type;
```

The default definition of `coroutine_traits` template defines the `promise_type` by looking for a nested `promise_type` typedef defined on the return-type. ie. Something like this (but with some extra SFINAE magic so that `promise_type` is not defined if `RET::promise_type` is not defined).

```
namespace std::experimental
{
  template<typename RET, typename... ARGS>
  struct coroutine_traits<RET, ARGS...>
  {
    using promise_type = typename RET::promise_type;
  };
}
```

So for coroutine return-types that you have control over, you can just define a nested `promise_type` in your class to have the compiler use that type as the type of the promise object for coroutines that return your class.

For example:

```
template<typename T>
struct task
{
  using promise_type = task_promise<T>;
  ...
};
```

However, for coroutine return-types that you don't have control over you can specialise the `coroutine_traits` to define the promise type to use without needing to modify the type.

For example, to define the promise-type to use for a coroutine that returns `std::optional<T>` :

```
namespace std::experimental
{
  template<typename T, typename... ARGS>
  struct coroutine_traits<std::optional<T>, ARGS...>
  {
    using promise_type = optional_promise<T>;
  };
}
```

Identifying a specific coroutine activation frame

When you call a coroutine function, a coroutine frame is created. In order to resume the associated coroutine or destroy the coroutine frame you need some way to identify or refer to that particular coroutine frame.

The mechanism the Coroutines TS provides for this is the `coroutine_handle` type.

The (abbreviated) interface of this type is as follows:

```
namespace std::experimental
{
  template<typename Promise = void>
  struct coroutine_handle;

  // Type-erased coroutine handle. Can refer to any kind of coroutine.
  // Doesn't allow access to the promise object.
  template<>
  struct coroutine_handle<void>
  {
    // Constructs to the null handle.
    constexpr coroutine_handle();

    // Convert to/from a void* for passing into C-style interop functions.
    constexpr void* address() const noexcept;
    static constexpr coroutine_handle from_address(void* addr);

    // Query if the handle is non-null.
    constexpr explicit operator bool() const noexcept;

    // Query if the coroutine is suspended at the final_suspend point.
    // Undefined behaviour if coroutine is not currently suspended.
    bool done() const;

    // Resume/Destroy the suspended coroutine
    void resume();
    void destroy();
  };

  // Coroutine handle for coroutines with a known promise type.
  // Template argument must exactly match coroutine's promise type.
  template<typename Promise>
  struct coroutine_handle : coroutine_handle<>
  {
    using coroutine_handle<>::coroutine_handle;

    static constexpr coroutine_handle from_address(void* addr);

    // Access to the coroutine's promise object.
    Promise& promise() const;

    // You can reconstruct the coroutine handle from the promise object.
    static coroutine_handle from_promise(Promise& promise);
  };
}
```

You can obtain a `coroutine_handle` for a coroutine in two ways:

1. It is passed to the `await_suspend()` method during a `co_await` expression.
2. If you have a reference to the coroutine's promise object, you can reconstruct its `coroutine_handle` using `coroutine_handle<Promise>::from_promise()` .

The `coroutine_handle` of the awaiting coroutine will be passed into the `await_suspend()` method of the awaiter after the coroutine has suspended at the `<suspend-point>` of a `co_await` expression. You can think of this `coroutine_handle` as representing the continuation of the coroutine in a continuation-passing style call.

Note that the `coroutine_handle` is **NOT** and RAII object. You must manually call `.destroy()` to destroy the coroutine frame and free its resources. Think of it as the equivalent of a `void*` used to manage memory. This is for performance reasons: making it an RAII object would add additional overhead to coroutine, such as the need for reference counting.

You should generally try to use higher-level types that provide the RAII semantics for coroutines, such as those provided by cppcoro (shameless plug), or write your own higher-level types that encapsulate the lifetime of the coroutine frame for your coroutine type.

Customising the behaviour of `co_await`

The promise type can optionally customise the behaviour of every `co_await` expression that appears in the body of the coroutine.

By simply defining a method named `await_transform()` on the promise type, the compiler will then transform every `co_await <expr>` appearing in the body of the coroutine into `co_await promise.await_transform(<expr>)` .

This has a number of important and powerful uses:

It lets you enable awaiting types that would not normally be awaitable.

For example, a promise type for coroutines with a `std::optional<T>` return-type might provide an `await_transform()` overload that takes a `std::optional<U>` and that returns an awaitable type that either returns a value of type `U` or suspends the coroutine if the awaited value contains `nullopt` .

```
template<typename T>
class optional_promise
{
  ...

  template<typename U>
  auto await_transform(std::optional<U>& value)
  {
    class awaiter
    {
      std::optional<U>& value;
    public:
      explicit awaiter(std::optional<U>& x) noexcept : value(x) {}
      bool await_ready() noexcept { return value.has_value(); }
      void await_suspend(std::experimental::coroutine_handle<>) noexcept {}
      U& await_resume() noexcept { return *value; }
    };
    return awaiter{ value };
  }
};
```

It lets you disallow awaiting on certain types by declaring `await_transform` overloads as deleted.

For example, a promise type for `std::generator<T>` return-type might declare a deleted `await_transform()` template member function that accepts any type. This basically disables use of `co_await` within the coroutine.

```
template<typename T>
class generator_promise
{
  ...

  // Disable any use of co_await within this type of coroutine.
  template<typename U>
  std::experimental::suspend_never await_transform(U&&) = delete;

};
```

It lets you adapt and change the behaviour of normally awaitable values

For example, you could define a type of coroutine that ensured that the coroutine always resumed from every `co_await` expression on an associated executor by wrapping the awaitable in a `resume_on()` operator (see `cppcoro::resume_on()`).

```
template<typename T, typename Executor>
class executor_task_promise
{
  Executor executor;

public:

  template<typename Awaitable>
  auto await_transform(Awaitable&& awaitable)
  {
    using cppcoro::resume_on;
    return resume_on(this->executor, std::forward<Awaitable>(awaitable));
  }
};
```

As a final word on `await_transform()`, it's important to note that if the promise type defines *any* `await_transform()` members then this triggers the compiler to transform *all* `co_await` expressions to call `promise.await_transform()`. This means that if you want to customise the behaviour of `co_await` for just some types that you also need to provide a fallback overload of `await_transform()` that just forwards through the argument.

Customising the behaviour of `co_yield`

The final thing you can customise through the promise type is the behaviour of the `co_yield` keyword.

If the `co_yield` keyword appears in a coroutine then the compiler translates the expression `co_yield <expr>` into the expression `co_await promise.yield_value(<expr>)`. The promise type can therefore customise the behaviour of the `co_yield` keyword by defining one or more `yield_value()` methods on the promise object.

Note that, unlike `await_transform`, there is no default behaviour of `co_yield` if the promise type does not define the `yield_value()` method. So while a promise type needs to explicitly opt-out of allowing `co_await` by declaring a deleted `await_transform()`, a promise type needs to opt-in to supporting `co_yield`.

The typical example of a promise type with a `yield_value()` method is that of a `generator<T>` type:

```
template<typename T>
class generator_promise
{
  T* valuePtr;
public:
  ...

  std::experimental::suspend_always yield_value(T& value) noexcept
  {
    // Stash the address of the yielded value and then return an awaitable
    // that will cause the coroutine to suspend at the co_yield expression.
    // Execution will then return from the call to coroutine_handle<>::resume()
    // inside either generator<T>::begin() or generator<T>::iterator::operator++().
    valuePtr = std::addressof(value);
    return {};
  }
};
```

Summary

In this post I've covered the individual transformations that the compiler applies to a function when compiling it as a coroutine.

Hopefully this post will help you to understand how you can customise the behaviour of different types of coroutines through defining different your own promise type. There are a lot of moving parts in the coroutine mechanics and so there are lots of different ways that you can customise their behaviour.

However, there is still one more important transformation that the compiler performs which I have not yet covered - the transformation of the coroutine body into a state-machine. However, this post is already too long so I will defer explaining this to the next post. Stay tuned!

C++ Coroutines: Understanding Symmetric Transfer

lewissbaker.github.io/2020/05/11/understanding_symmetric_transfer

Asymmetric Transfer 11 May 2020

May 11, 2020

The Coroutines TS provided a wonderful way to write asynchronous code as if you were writing synchronous code. You just need to sprinkle `co_await` at appropriate points and the compiler takes care of suspending the coroutine, preserving state across suspend-points and resuming execution of the coroutine later when the operation completes.

However, the Coroutines TS, as it was originally specified, had a nasty limitation that could easily lead to stack-overflow if you weren't careful. And if you wanted to avoid this stack-overflow then you had to introduce extra synchronisation overhead to safely guard against this in your `task<T>` type.

Thankfully, a tweak was made to the design of coroutines in 2018 to add a capability called "symmetric transfer" which allows you to suspend one coroutine and resume another coroutine without consuming any additional stack-space. The addition of this capability lifted a key limitation of the Coroutines TS and allows for much simpler and more efficient implementation of async coroutine types without sacrificing any of the safety aspects needed to guard against stack-overflow.

In this post I will attempt to explain the stack-overflow problem and how the addition of this key "symmetric transfer" capability lets us solve this problem.

First some background on how a task coroutine works

Consider the following coroutines:

```
task foo() {
  co_return;
}

task bar() {
  co_await foo();
}
```

Assume we have a simple `task` type that lazily executes the body when another coroutine awaits it. This particular `task` type does not support returning a value.

Let's unpack what's happening here when `bar()` evaluates `co_await foo()`.

- The `bar()` coroutine calls the `foo()` function. Note that from the caller's perspective a coroutine is just an ordinary function.

- The invocation of `foo()` performs a few steps:
  - Allocates storage for a coroutine frame (typically on the heap)
  - Copies parameters into the coroutine frame (in this case there are no parameters so this is a no-op).
  - Constructs the promise object in the coroutine frame
  - Calls `promise.get_return_object()` to get the return-value for `foo()`. This produces the `task` object that will be returned, initialising it with a `std::coroutine_handle` that refers to the coroutine frame that was just created.
  - Suspends execution of the coroutine at the initial-suspend point (ie. the open curly brace)
  - Returns the `task` object back to `bar()`.
- Next the `bar()` coroutine evaluates the `co_await` expression on the `task` returned from `foo()`.
  - The `bar()` coroutine suspends and then calls the `await_suspend()` method on the returned task, passing it the `std::coroutine_handle` that refers to `bar()`'s coroutine frame.
  - The `await_suspend()` method then stores `bar()`'s `std::coroutine_handle` in `foo()`'s promise object and then resumes the `foo()` coroutine by calling `.resume()` on `foo()`'s `std::coroutine_handle`.
- The `foo()` coroutine executes and runs to completion synchronously.
- The `foo()` coroutine suspends at the final-suspend point (ie. the closing curly brace) and then resumes the coroutine identified by the `std::coroutine_handle` that was stored in its promise object before it was started. ie. `bar()`'s coroutine.
- The `bar()` coroutine resumes and continues executing and eventually reaches the end of the statement containing the `co_await` expression at which point it calls the destructor of the temporary `task` object returned from `foo()`.
- The `task` destructor then calls the `.destroy()` method on `foo()`'s coroutine handle which then destroys the coroutine frame along with the promise object and copies of any arguments.

Ok, so that seems like a lot of steps for a simple call.

To help understand this in a bit more depth, let's look at how a naive implementation of this `task` class would look when implemented using the the Coroutines TS design (which didn't support symmetric transfer).

Outline of a `task` implementation

The outline of the class looks something like this:

```
class task {
public:
  class promise_type { /* see below */ };

  task(task&& t) noexcept
  : coro_(std::exchange(t.coro_, {}))
  {}

  ~task() {
    if (coro_)
      coro_.destroy();
  }

  class awaiter { /* see below */ };

  awaiter operator co_await() && noexcept;

private:
  explicit task(std::coroutine_handle<promise_type> h) noexcept
  : coro_(h)
  {}

  std::coroutine_handle<promise_type> coro_;
};
```

A `task` has exclusive ownership of the `std::coroutine_handle` that corresponds to the coroutine frame created during the invocation of the coroutine. The `task` object is an RAII object that ensures that `.destroy()` is called on the `std::coroutine_handle` when the `task` object goes out of scope.

So now let's expand on the `promise_type` .

Implementing `task::promise_type`

From the previous post we know that the `promise_type` member defines the type of the **Promise** object that is created within the coroutine frame and that controls the behaviour of the coroutine.

First, we need to implement the `get_return_object()` to construct the `task` object to return when the coroutine is invoked. This method just needs to initialise the task with the `std::coroutine_handle` of the newly create coroutine frame.

We can use the `std::coroutine_handle::from_promise()` method to manufacture one of these handles from the promise object.

```
class task::promise_type {
public:
  task get_return_object() noexcept {
    return task{std::coroutine_handle<promise_type>::from_promise(*this)};
  }
```

Next, we want the coroutine to initially suspend at the open curly brace so that we can later resume the coroutine from this point when the returned `task` is awaited.

There are several benefits of starting the coroutine lazily:

1. It means that we can attach the continuation's `std::coroutine_handle` before starting execution of the coroutine. This means we don't need to use thread-synchronisation to arbitrate the race between attaching the continuation later and the coroutine running to completion.
2. It means that the `task` destructor can unconditionally destroy the coroutine frame - we don't need to worry about whether the coroutine is potentially executing on another thread since the coroutine will not start executing until we await it, and while it is executing the calling coroutine is suspended and so won't attempt to call the task destructor until the coroutine finishes executing. This gives the compiler a much better chance at inlining the allocation of the coroutine frame into the frame of the caller. See P0981R0 to read more about the Heap Allocation eLision Optimisation (HALO).
3. It also improves the exception-safety of your coroutine code. If you don't immediately `co_await` the returned `task` and do something else that can throw an exception that causes the stack to unwind and the `task` destructor to run then we can safely destroy the coroutine since we know it hasn't started yet. We aren't left with the difficult choice between detaching, potentially leaving dangling references, blocking in the destructor, terminating or undefined-behaviour. This is something that I cover in a bit more detail in my CppCon 2019 talk on Structured Concurrency.

To have the coroutine initially suspend at the open curly brace we define an `initial_suspend()` method that returns the builtin `suspend_always` type.

```
std::suspend_always initial_suspend() noexcept {
  return {};
}
```

Next, we need to define the `return_void()` method, called when you execute `co_return;` or when execution runs off the end of the coroutine. This method doesn't actually need to do anything, it just needs to exist so that the compiler knows that `co_return;` is valid within this coroutine type.

```
void return_void() noexcept {}
```

We also need to add an `unhandled_exception()` method that is called if an exception escapes the body of the coroutine. For our purposes we can just treat the task coroutine bodies as `noexcept` and call `std::terminate()` if this happens.

```
void unhandled_exception() noexcept {
  std::terminate();
}
```

Finally, when the coroutine execution reaches the closing curly brace, we want the coroutine to suspend at the final-suspend point and then resume its continuation. ie. the coroutine that is awaiting the completion of this coroutine.

To support this, we need a data-member in the promise to hold the `std::coroutine_handle` of the continuation. We also need to define the `final_suspend()` method that returns an awaitable object that will resume the continuation after the current coroutine has suspended at the final-suspend point.

It's important to delay resuming the continuation until after the current coroutine has suspended because the continuation may go on to immediately call the `task` destructor which will call `.destroy()` on the coroutine frame. The `.destroy()` method is only valid to call on a suspended

coroutine and so it would be undefined-behaviour to resume the continuation before the current coroutine has suspended.

The compiler inserts code to evaluate the statement `co_await promise.final_suspend();` at the closing curly brace.

It's important to note that the coroutine is not yet in a suspended state when the `final_suspend()` method is invoked. We need to wait until the `await_suspend()` method on the returned awaitable is called before the coroutine is suspended.

```
  struct final_awaiter {
    bool await_ready() noexcept {
      return false;
    }

    void await_suspend(std::coroutine_handle<promise_type> h) noexcept {
      // The coroutine is now suspended at the final-suspend point.
      // Lookup its continuation in the promise and resume it.
      h.promise().continuation.resume();
    }

    void await_resume() noexcept {}
  };

  final_awaiter final_suspend() noexcept {
    return {};
  }

  std::coroutine_handle<> continuation;
};
```

Ok, so that's the complete `promise_type`. The final piece we need to implement is the `task::operator co_await()`.

Implementing `task::operator co_await()`

You may remember from the Understanding operator co_await() post that when evaluating a `co_await` expression, the compiler will generate a call to `operator co_await()`, if one is defined, and then the object returned must have the `await_ready()`, `await_suspend()` and `await_resume()` methods defined.

When a coroutine awaits a `task` we want the awaiting coroutine to always suspend and then, once it has suspended, store the awaiting coroutine's handle in the promise of the coroutine we are about to resume and then call `.resume()` on the `task`'s `std::coroutine_handle` to start executing the task.

Thus the relatively straight forward code:

```
class task::awaiter {
public:
  bool await_ready() noexcept {
    return false;
  }

  void await_suspend(std::coroutine_handle<> continuation) noexcept {
    // Store the continuation in the task's promise so that the final_suspend()
    // knows to resume this coroutine when the task completes.
    coro_.promise().continuation = continuation;

    // Then we resume the task's coroutine, which is currently suspended
    // at the initial-suspend-point (ie. at the open curly brace).
    coro_.resume();
  }

  void await_resume() noexcept {}

private:
  explicit awaiter(std::coroutine_handle<task::promise_type> h) noexcept
  : coro_(h)
  {}

  std::coroutine_handle<task::promise_type> coro_;
};

task::awaiter task::operator co_await() && noexcept {
  return awaiter{coro_};
}
```

And thus completes the code necessary for a functional `task` type.

You can see the complete set of code in Compiler Explorer here: https://godbolt.org/z/-Kw6Nf

The stack-overflow problem

The limitation of this implementation arises, however, when you start writing loops within your coroutines and you `co_await` tasks that can potentially complete synchronously within the body of that loop.

For example:

```
task completes_synchronously() {
  co_return;
}

task loop_synchronously(int count) {
  for (int i = 0; i < count; ++i) {
    co_await completes_synchronously();
  }
}
```

With the naive `task` implementation described above, the `loop_synchronously()` function will (probably) work fine when `count` is 10, 1000, or even 100'000. But there will be a value that you can pass that will eventually cause this coroutine to start crashing.

For example, see: https://godbolt.org/z/gy5Q8q which crashes when `count` is 1'000'000.

The reason that this is crashing is because of stack-overflow.

To understand why this code is causing a stack-overflow we need to take a look at what is happening when this code is executing. In particular, what is happening to the stack-frames.

When the `loop_synchronously()` coroutine first starts executing it will be because some other coroutine `co_await` ed the `task` returned. This will in turn suspend the awaiting coroutine and call `task::awaiter::await_suspend()` which will call `resume()` on the task's `std::coroutine_handle` .

Thus the stack will look something like this when `loop_synchronously()` starts:

```
          Stack                                                  Heap
+-----------------------------+  <-- top of stack   +-------------------------+
| loop_synchronously$resume   | active coroutine -> | loop_synchronously frame |
+-----------------------------+                     | +---------------------+ |
| coroutine_handle::resume    |                     | | task::promise       | |
+-----------------------------+                     | | - continuation --.  | |
| task::awaiter::await_suspend |                    | +------------------|---+ |
+-----------------------------+                     | ...                |     |
| awaiting_coroutine$resume   |                     +--------------------|-----+
+-----------------------------+                                          V
|  ....                       |                     +-------------------------+
+-----------------------------+                     | awaiting_coroutine frame |
                                                    |                         |
                                                    +-------------------------+
```

> Note: When a coroutine function is compiled the compiler typically splits it into two parts:
>
> 1. the "ramp function" which deals with the construction of the coroutine frame, parameter copying, promise construction and producing the return-value, and
> 2. the "coroutine body" which contains the user-authored logic from the body of the coroutine.
>
> I use the `$resume` suffix to refer to the "coroutine body" part of the coroutine.
>
> A later blog post will go into more detail about this split.

Then when `loop_synchronously()` awaits the `task` returned from `completes_synchronously()` the current coroutine is suspended and calls `task::awaiter::await_suspend()` . The `await_suspend()` method then calls `.resume()` on the coroutine handle corresponding to the `completes_synchronously()` coroutine.

This resumes the `completes_synchronously()` coroutine which then runs to completion synchronously and suspends at the final-suspend point. It then calls `task::promise::final_awaiter::await_suspend()` which calls `.resume()` on the coroutine handle corresponding to `loop_synchronously()` .

The net result of all of this is that if we look at the state of the program just after the `loop_synchronously()` coroutine is resumed and just before the temporary `task` returned by `completes_synchronously()` is destroyed at the semicolon then the stack/heap should look something like this:

```
          Stack                                                Heap
+-------------------------------+ <-- top of stack
| loop_synchronously$resume     | active coroutine -.
+-------------------------------+                   |
| coroutine_handle::resume      |            .------'
+-------------------------------+            |
| final_awaiter::await_suspend  |            |
+-------------------------------+            |  +-------------------------+ <-.
| completes_synchronously$resume|            |  | completes_synchronously  |   |
+-------------------------------+            |  | frame                    |   |
| coroutine_handle::resume      |            |  +-------------------------+   |
+-------------------------------+            '---.                            |
| task::awaiter::await_suspend  |                V                            |
+-------------------------------+ <-- prev top  +-------------------------+   |
| loop_synchronously$resume     |     of stack  | loop_synchronously frame |   |
+-------------------------------+               | +----------------------+ |   |
| coroutine_handle::resume      |               | | task::promise        | |   |
+-------------------------------+               | | - continuation --.   | |   |
| task::awaiter::await_suspend  |               | +------------------|---+ |   |
+-------------------------------+               | - task temporary --|---------'
| awaiting_coroutine$resume     |               +--------------------|-----+
+-------------------------------+                                    V
|  ....                         |               +-------------------------+
+-------------------------------+               | awaiting_coroutine frame |
                                                |                         |
                                                +-------------------------+
```

Then the next thing this will do is call the `task` destructor which will destroy the `completes_synchronously()` frame. It will then increment the `count` variable and go around the loop again, creating a new `completes_synchronously()` frame and resuming it.

In effect, what is happening here is that `loop_synchronously()` and `completes_synchronously()` end up recursively calling each other. Each time this happens we end up consuming a bit more stack-space, until eventually, after enough iterations, we overflow the stack and end up in undefined-behaviour land, typically resulting in your program promptly crashing.

Writing loops in coroutines built this way makes it very easy to write functions that perform unbounded recursion without looking like they are doing any recursion.

So, what would the solution look like under the original Coroutines TS design?

The Coroutines TS solution

Ok, so what can we do about this to avoid this kind of unbounded recursion?

With the above implementation we are using the variant of `await_suspend()` that returns `void`. In the Coroutines TS there is also a version of `await_suspend()` that returns `bool` - if it returns `true` then the coroutine is suspended and execution returns to the caller of `resume()`, otherwise if it returns `false` then the coroutine is immediately resumed, but this time without consuming any additional stack-space.

So, to avoid the unbounded mutual recursion what we want to do is make use of the `bool`-returning version of `await_suspend()` to resume the current coroutine by returning `false` from the `task::awaiter::await_suspend()` method if the task completes synchronously instead of resuming the coroutine recursively using `std::coroutine_handle::resume()`.

To implement a general solution for this there are two parts.

1. Inside the `task::awaiter::await_suspend()` method you can start executing the coroutine by calling `.resume()`. Then when the call to `.resume()` returns, check whether the coroutine has run to completion or not. If it has run to completion then we can return `false`, which indicates the awaiting coroutine should immediately resume, or we can return `true`, indicating that execution should return to the caller of `std::coroutine_handle::resume()`.

2. Inside `task::promise_type::final_awaiter::await_suspend()`, which is run when the coroutine runs to completion, we need to check whether the awaiting coroutine has (or will) return `true` from `task::awaiter::await_suspend()` and if so then resume it by calling `.resume()`. Otherwise, we need to avoid resuming the coroutine and notify `task::awaiter::await_suspend()` that it needs to return `false`.

There is an added complication, however, in that it's possible for a coroutine to start executing on the current thread then suspend and later resume and run to completion on a different thread before the call to `.resume()` returns. Thus, we need to be able to resolve the potential race between part 1 and part 2 above happening concurrently.

We will need to use a `std::atomic` value to decide the winner of the race here.

Now for the code. We can make the following modifications:

```
class task::promise_type {
  ...

  std::coroutine_handle<> continuation;
  std::atomic<bool> ready = false;
};

bool task::awaiter::await_suspend(
    std::coroutine_handle<> continuation) noexcept {
  promise_type& promise = coro_.promise();
  promise.continuation = continuation;
  coro_.resume();
  return !promise.ready.exchange(true, std::memory_order_acq_rel);
}

void task::promise_type::final_awaiter::await_suspend(
    std::coroutine_handle<promise_type> h) noexcept {
  promise_type& promise = h.promise();
  if (promise.ready.exchange(true, std::memory_order_acq_rel)) {
    // The coroutine did not complete synchronously, resume it here.
    promise.continuation.resume();
  }
}
```

See the updated example on Compiler Explorer: https://godbolt.org/z/7fm8Za Note how it no longer crashes when executing the `count == 1'000'000` case.

This turns out to be the approach that the `cppcoro::task<T>` implementation took to avoid the unbounded recursion problem (and still does for some platforms) and it has worked reasonably well.

Woohoo! Problem solved, right? Ship it! Right...?

The problems

While the above solution does solve the recursion problem it has a couple of drawbacks.

Firstly, it introduces the need for `std::atomic` operations which can be quite costly. There is an atomic exchange on the caller when suspending the awaiting coroutine, and another atomic exchange on the callee when it runs to completion. If your application only ever executes on a single thread then you are paying the cost of the atomic operations for synchronising threads even though it's never needed.

Secondly, it introduces additional branches. One in the caller, which needs to decide whether to suspend or immediately resume the coroutine, and one in the callee, which needs to decide whether to resume the continuation or suspend.

Note that the cost of this extra branch, and possibly even the atomic operations, would often be dwarfed by the cost of the business logic present in the coroutine. However, coroutines have been advertised as a zero cost abstraction and there have even been people using coroutines to suspend execution of a function to avoid waiting for an L1-cache-miss (see Gor's great CppCon talk on nanocoroutines for more details on this).

Thirdly, and probably most importantly, it introduces some non-determinism in the execution-context that the awaiting coroutine resumes on.

Let's say I have the following code:

```
cppcoro::static_thread_pool tp;

task foo()
{
  std::cout << "foo1 " << std::this_thread::get_id() << "\n";
  // Suspend coroutine and reschedule onto thread-pool thread.
  co_await tp.schedule();
  std::cout << "foo2 " << std::this_thread::get_id() << "\n";
}

task bar()
{
  std::cout << "bar1 " << std::this_thread::get_id() << "\n";
  co_await foo();
  std::cout << "bar2" << std::this_thread::get_id() << "\n";
}
```

With the original implementation we were guaranteed that the code that runs after `co_await foo()` would run inline on the same thread that `foo()` completed on.

For example, one possible output would have been:

```
bar1 1234
foo1 1234
foo2 3456
bar2 3456
```

However, with the changes to use atomics, it's possible the completion of `foo()` may race with the suspension of `bar()` and this can, in some cases, mean that the code after `co_await foo()` might run on the original thread that `bar()` started executing on.

For example, the following output might now also be possible:

```
bar1 1234
foo1 1234
foo2 3456
bar2 1234
```

For many use-cases this behaviour may not make a difference. However, for algorithms whose purpose is to transition execution context this can be problematic.

For example, the `via()` algorithm awaits some Awaitable and then produces it on the specified scheduler's execution context. A simplified version of this algorithm is shown below.

```
template<typename Awaitable, typename Scheduler>
task<await_result_t<Awaitable>> via(Awaitable a, Scheduler s)
{
  auto result = co_await std::move(a);
  co_await s.schedule();
  co_return result;
}

task<T> get_value();
void consume(const T&);

task<void> consumer(static_thread_pool::scheduler s)
{
  T result = co_await via(get_value(), s);
  consume(result);
}
```

With the original version the call to `consume()` is always guaranteed to be executed on the thread-pool, `s`. However, with the revised version that uses atomics it's possible that `consume()` might either be executed on a thread associated with the scheduler, `s`, or on whatever thread the `consumer()` coroutine started execution on.

So how do we solve the stack-overflow problem without the overhead of the atomic operations, extra branches and the non-deterministic resumption context?

Enter "symmetric transfer"!

The paper P0913R0 "Add symmetric coroutine control transfer" by Gor Nishanov (2018) proposed a solution to this problem by providing a facility which allows one coroutine to suspend and then resume another coroutine symmetrically without consuming any additional stack-space.

This paper proposed two key changes:

- Allow returning a `std::coroutine_handle<T>` from `await_suspend()` as a way of indicating that execution should be symmetrically transferred to the coroutine identified by the returned handle.
- Add a `std::experimental::noop_coroutine()` function that returns a special `std::coroutine_handle` that can be returned from `await_suspend()` to suspend the current coroutine and return from the call to `.resume()` instead of transferring execution to another coroutine.

So what do we mean by "symmetric transfer"?

When you resume a coroutine by calling `.resume()` on it's `std::coroutine_handle` the caller of `.resume()` remains active on the stack while the resumed coroutine executes. When this coroutine next suspends and the call to `await_suspend()` for that suspend-point returns either `void` (indicating unconditional suspend) or `true` (indicating conditional suspend) then call to `.resume()` will return.

This can be thought of as an “asymmetric transfer” of execution to the coroutine and behaves just like an ordinary function call. The caller of `.resume()` can be any function (which may or may not be a coroutine). When that coroutine suspends and returns either `true` or `void` from `await_suspend()` then execution will return from the call to `.resume()` and

Every time we resume a coroutine by calling `.resume()` we create a new stack-frame for the execution of that coroutine.

However, with “symmetric transfer” we are simply suspending one coroutine and resuming another coroutine. There is no implicit caller/callee relationship between the two coroutines - when a coroutine suspends it can transfer execution to any suspended coroutine (including itself) and does not necessarily have to transfer execution back to the previous coroutine when it next suspends or completes.

Let’s look at what the compiler lowers a `co_await` expression to when the awaiter makes use of symmetric-transfer:

```
{
  decltype(auto) value = <expr>;
  decltype(auto) awaitable =
      get_awaitable(promise, static_cast<decltype(value)&&>(value));
  decltype(auto) awaiter =
      get_awaiter(static_cast<decltype(awaitable)&&>(awaitable));
  if (!awaiter.await_ready())
  {
    using handle_t = std::coroutine_handle<P>;

    //<suspend-coroutine>

    auto h = awaiter.await_suspend(handle_t::from_promise(p));
    h.resume();
    //<return-to-caller-or-resumer>

    //<resume-point>
  }

  return awaiter.await_resume();
}
```

Let’s zoom in on the key part that differs from other `co_await` forms:

```
auto h = awaiter.await_suspend(handle_t::from_promise(p));
h.resume();
//<return-to-caller-or-resumer>
```

Once the coroutine state-machine is lowered (a topic for another post), the `<return-to-caller-or-resumer>` part basically becomes a `return;` statement which causes the call to `.resume()` that last resumed the coroutine to return to its caller.

This means that we have the situation where we have a call to another function with the same signature, `std::coroutine_handle::resume()` , followed by a `return;` from the current function which is itself the body of a `std::coroutine_handle::resume()` call.

Some compilers, when optimisations are enabled, are able to apply an optimisation that turns calls to other functions the tail-position (ie. just before returning) into tail-calls as long as some conditions are met.

It just so happens that this kind of tail-call optimisation is exactly the kind of thing we want to be able to do to avoid the stack-overflow problem we were encountering before. But instead of being at the mercy of the optimiser as to whether or not the tail-call transformation is perfromed, we want to be able to guarantee that the tail-call transformation occurs, even when optimisations are not enabled.

But first let's dig into what we mean by tail-calls.

Tail-calls

A tail-call is one where the current stack-frame is popped before the call and the current function's return address becomes the return-address for the callee. ie. the callee will return directly the the caller of this function.

On X86/X64 architectures this generally means that the compiler will generate code that first pops the current stack-frame and then uses a `jmp` instruction to jump to the called function's entry-point instead of using a `call` instruction and then popping the current stack-frame after the `call` returns.

This optimisation is generally only possible to do in limited circumstances, however.

In particular, it requires that:

- the calling convention supports tail-calls and is the same for the caller and callee;
- the return-type is the same;
- there are no non-trivial destructors that need to be run after the call before returning to the caller; and
- the call is not inside a try/catch block.

The shape of the symmetric-transfer form of `co_await` has actually been designed specifically to allow coroutines to satisfy all of these requirements. Let's look at them individually.

Calling convention When the compiler lowers a coroutine into machine code it actually splits the coroutine up into two parts: the ramp (which allocates and initialises the coroutine frame) and the body (which contains the state-machine for the user-authored coroutine body).

The function signature of the coroutine (and thus any user-specified calling-convention) affects only the ramp part, whereas the body part is under the control of the compiler and is never directly called by any user-code - only by the ramp function and by `std::coroutine_handle::resume()` .

The calling-convention of the coroutine body part is not user-visible and is entirely up to the compiler and thus it can choose an appropriate calling convention that supports tail-calls and that is used by all coroutine bodies.

Return type is the same The return-type for both the source and target coroutine's `.resume()` method is `void` so this requirement is trivially satisfied.

No non-trivial destructors When performing a tail-call we need to be able to free the current stack-frame before calling the target function and this requires the lifetime of all stack-allocated objects to have ended prior to the call.

Normally, this would be problematic as soon as there are any objects with non-trivial destructors in-scope as the lifetime of those objects would not yet have ended and those objects would have been allocated on the stack.

However, when a coroutine suspends it does so without exiting any scopes and the way it achieves this is by placing any objects whose lifetime spans a suspend-point in the coroutine frame rather than allocating them on the stack.

Local variables with lifetimes that do not span a suspend-point may be allocated on the stack, but the lifetime of these objects will have already ended and their destructors will have been called before the coroutine next suspends.

Thus there should be no non-trivial destructors for stack-allocated objects that need to be run after the return of the tail-call.

Call not inside a try/catch block This one is a little tricker as within every coroutine there is an implicit try/catch block that encloses the user-authored body of the coroutine.

From the specification, we see that the coroutine is defined as:

```
{
  promise_type promise;
  co_await promise.initial_suspend();
  try { F; }
  catch (...) { promise.unhandled_exception(); }
final_suspend:
  co_await promise.final_suspend();
}
```

Where `F` is the user-authored part of the coroutine body.

Thus every user-authored `co_await` expression (other than initial/final_suspend) exists within the context of a try/catch block.

However, implementations work around this by actually executing the call to `.resume()` *outside* of the context of the try-block.

I hope to be able to go into this aspect in more detail in another blog post that goes into the details of the lowering of a coroutine to machine-code (this post is already long enough).

> Note, however, that the current wording in the C++ specification is not clear on requiring implementations to do this and it is only a non-normative note that hints that this is something that might be required. Hopefully we'll be able to fix the specification in the future.

So we see that coroutines performing a symmetric-transfer generally satisfy all of the requirements for being able to perform a tail-call. The compiler guarantees that this will always be a tail-call, regardless of whether optimisations are enabled or not.

This means that by using the `std::coroutine_handle` -returning flavour of `await_suspend()` we can suspend the current coroutine and transfer execution to another coroutine without consuming extra stack-space.

This allows us to write coroutines that mutually and recursively resume each other to an arbitrary depth without fear of overflowing the stack.

This is exactly what we need to fix our `task` implementation.

`task` revisited

So with the new "symmetric transfer" capability under our belt let's go back and fix our `task` type implementation.

To do this we need to make changes to the two `await_suspend()` methods in our implementation:

- First so that when we await the task that we perform a symmetric-transfer to resume the task's coroutine.
- Second so that when the task's coroutine completes that it performs a symmetric transfer to resume the awaiting coroutine.

To address the await direction we need to change the `task::awaiter` method from this:

```
void task::awaiter::await_suspend(
    std::coroutine_handle<> continuation) noexcept {
  // Store the continuation in the task's promise so that the final_suspend()
  // knows to resume this coroutine when the task completes.
  coro_.promise().continuation = continuation;

  // Then we resume the task's coroutine, which is currently suspended
  // at the initial-suspend-point (ie. at the open curly brace).
  coro_.resume();
}
```

to this:

```
std::coroutine_handle<> task::awaiter::await_suspend(
    std::coroutine_handle<> continuation) noexcept {
  // Store the continuation in the task's promise so that the final_suspend()
  // knows to resume this coroutine when the task completes.
  coro_.promise().continuation = continuation;

  // Then we tail-resume the task's coroutine, which is currently suspended
  // at the initial-suspend-point (ie. at the open curly brace), by returning
  // its handle from await_suspend().
  return coro_;
}
```

And to address the return-path we need to update the `task::promise_type::final_awaiter` method from this:

```
void task::promise_type::final_awaiter::await_suspend(
    std::coroutine_handle<promise_type> h) noexcept {
  // The coroutine is now suspended at the final-suspend point.
  // Lookup its continuation in the promise and resume it.
  h.promise().continuation.resume();
}
```

to this:

```
std::coroutine_handle<> task::promise_type::final_awaiter::await_suspend(
    std::coroutine_handle<promise_type> h) noexcept {
  // The coroutine is now suspended at the final-suspend point.
  // Lookup its continuation in the promise and resume it symmetrically.
  return h.promise().continuation;
}
```

And now we have a `task` implementation that doesn't suffer from the stack-overflow problem that the `void`-returning `await_suspend` flavour had and that doesn't have the non-deterministic resumption context problem of the `bool`-returning `await_suspend` flavour had.

Visualising the stack

Let's now go back and have a look at our original example:

```
task completes_synchronously() {
  co_return;
}

task loop_synchronously(int count) {
  for (int i = 0; i < count; ++i) {
    co_await completes_synchronously();
  }
}
```

When the `loop_synchronously()` coroutine first starts executing it will be because some other coroutine `co_await`ed the `task` returned. This will have been launched by symmetric transfer from some other coroutine, which would have been resumed by a call to `std::coroutine_handle::resume()`.

Thus the stack will look something like this when `loop_synchronously()` starts:

```
          Stack                                                Heap
+---------------------------+  <-- top of stack   +-------------------------+
| loop_synchronously$resume | active coroutine -> | loop_synchronously frame |
+---------------------------+                     | +----------------------+ |
| coroutine_handle::resume  |                     | | task::promise        | |
+---------------------------+                     | | - continuation --.   | |
|     ...                   |                     | +------------------|---+ |
+---------------------------+                     | ...                |     |
                                                  +--------------------|-----+
                                                                       V
                                                  +-------------------------+
                                                  | awaiting_coroutine frame |
                                                  |                         |
                                                  +-------------------------+
```

Now, when it executes `co_await completes_synchronously()` it will perform a symmetric transfer to `completes_synchronously` coroutine.

It does this by:

- calling the `task::operator co_await()` which then returns the `task::awaiter` object
- then suspends and calls `task::awaiter::await_suspend()` which then returns the `coroutine_handle` of the `completes_synchronously` coroutine.
- then performs a tail-call / jump to `completes_synchronously` coroutine. This pops the `loop_synchronously` frame before activing the `completes_synchronously` frame.

If we now look at the stack just after `completes_synchronously` is resumed it will now look like this:

```
             Stack                                          Heap
                                              .-> +-------------------------+ <-.
                                              |   | completes_synchronously  |   |
                                              |   | frame                    |   |
                                              |   | +----------------------+ |   |
                                              |   | | task::promise        | |   |
                                              |   | | - continuation --.   | |   |
                                              |   | +------------------|---+ |   |
                                              `-, +--------------------|-----+   |
                                                |                      V         |
+------------------------------+ <-- top of  | +-------------------------+   |
| completes_synchronously$resume|     stack   | | loop_synchronously frame |   |
+------------------------------+ active -----' | +----------------------+ |   |
| coroutine_handle::resume      | coroutine     | | task::promise        | |   |
+------------------------------+               | | - continuation --.   | |   |
|     ...                       |               | +------------------|---+ |   |
+------------------------------+               | task temporary     |     |   |
                                               | - coro_       -----|---------`
                                               +--------------------|-----+
                                                                    V
                                               +-------------------------+
                                               | awaiting_coroutine frame |
                                               |                         |
                                               +-------------------------+
```

Note that the number of stack-frames has not grown here.

After the `completes_synchronously` coroutine completes and execution reaches the closing curly brace it will evaluate `co_await promise.final_suspend()` .

This will suspend the coroutine and call `final_awaiter::await_suspend()` which return the continuation's `std::coroutine_handle` (ie. the handle that points to the `loop_synchronously` coroutine). This will then do a symmetric transfer/tail-call to resume the `loop_synchronously` coroutine.

If we look at the stack just after `loop_synchronously` is resumed then it will look something like this:

```
           Stack                                                Heap
                                                  +-------------------------+ <-.
                                                  | completes_synchronously |   |
                                                  | frame                   |   |
                                                  | +---------------------+ |   |
                                                  | | task::promise       | |   |
                                                  | | - continuation --.  | |   |
                                                  | +------------------|---+ |   |
                                                  +--------------------|-----+   |
                                                                       V         |
+----------------------------+  <-- top of stack  +-------------------------+   |
| loop_synchronously$resume  | active coroutine -> | loop_synchronously frame |   |
+----------------------------+                    | +---------------------+ |   |
| coroutine_handle::resume() |                    | | task::promise       | |   |
+----------------------------+                    | | - continuation --.  | |   |
|     ...                    |                    | +------------------|---+ |   |
+----------------------------+                    | task temporary     |     |   |
                                                  | - coro_       -----|---------`
                                                  +--------------------|-----+
                                                                       V
                                                  +-------------------------+
                                                  | awaiting_coroutine frame |
                                                  |                         |
                                                  +-------------------------+
```

The first thing the `loop_synchronously` coroutine is going to do once resumed is to call the destructor of the temporary `task` that was returned from the call to `completes_synchronously` when execution reaches the semicolon. This will destroy the coroutine-frame, freeing its memory and leaving us with the following sitution:

```
           Stack                                                Heap
+----------------------------+  <-- top of stack  +-------------------------+
| loop_synchronously$resume | active coroutine -> | loop_synchronously frame |
+----------------------------+                    | +---------------------+ |
| coroutine_handle::resume  |                    | | task::promise       | |
+----------------------------+                    | | - continuation --.  | |
|     ...                    |                    | +------------------|---+ |
+----------------------------+                    | ...                |     |
                                                  +--------------------|-----+
                                                                       V
                                                  +-------------------------+
                                                  | awaiting_coroutine frame |
                                                  |                         |
                                                  +-------------------------+
```

We are now back to executing the `loop_synchronously` coroutine and we now have the same number of stack-frames and coroutine-frames as we started, and will do so each time we go around the loop.

Thus we can perform as many iterations of the loop as we want and will only use a constant amount of storage space.

For a full example of the symmetric-transfer version of the `task` type see the following Compiler Explorer link: https://godbolt.org/z/9baieF.

Symmetric Transfer as the Universal Form of await_suspend

Now that we see the power and importance of the symmetric-transfer form of the awaitable concept, I want to show you that this form is actually the universal form, which can theoretically replace the `void` and `bool` -returning forms of `await_suspend()` .

But first we need to look at the other piece that the P0913R0 proposal added to the coroutines design: `std::noop_coroutine()` .

Terminating the recursion

With the symmetric-transfer form of coroutines, every time the coroutine suspends it symmetrically resumes another coroutine. This is great as long as you have another coroutine to resume, but sometimes we don't have another coroutine to execute and just need to suspend and let execution return to the caller of `std::coroutine_handle::resume()` .

Both the `void` -returning and `bool` -returning flavours of `await_suspend()` allow a coroutine to suspend and return from `std::coroutine_handle::resume()` , so how do we do that with the symmetric-transfer flavour?

The answer is by using the special builtin `std::coroutine_handle` , called the "noop coroutine handle" which is produced by the function `std::noop_coroutine()` .

The "noop coroutine handle" is named as such because its `.resume()` implementation is such that it just immediately returns. i.e. resuming the coroutine is a no-op. Typically its implementation contains a single `ret` instruction.

If the `await_suspend()` method returns the `std::noop_coroutine()` handle then instead of transferring execution to the next coroutine, it transfers execution back to the caller of `std::coroutine_handle::resume()` .

Representing the other flavours of `await_suspend()`

With this information in-hand we can now show how to represent the other flavours of `await_suspend()` using the symmetric-transfer form.

The `void` -returning form

```
void my_awaiter::await_suspend(std::coroutine_handle<> h) {
  this->coro = h;
  enqueue(this);
}
```

can also be written using both the `bool` -returning form:

```
bool my_awaiter::await_suspend(std::coroutine_handle<> h) {
  this->coro = h;
  enqueue(this);
  return true;
}
```

and can be written using the symmetric-transfer form:

```
std::noop_coroutine_handle my_awaiter::await_suspend(
    std::coroutine_handle<> h) {
  this->coro = h;
  enqueue(this);
  return std::noop_coroutine();
}
```

The `bool` -returning form:

```
bool my_awaiter::await_suspend(std::coroutine_handle<> h) {
  this->coro = h;
  if (try_start(this)) {
    // Operation will complete asynchronously.
    // Return true to transfer execution to caller of
    // coroutine_handle::resume().
    return true;
  }

  // Operation completed synchronously.
  // Return false to immediately resume the current coroutine.
  return false;
}
```

can also be written using the symmetric-transfer form:

```
std::coroutine_handle<> my_awaiter::await_suspend(std::coroutine_handle<> h) {
  this->coro = h;
  if (try_start(this)) {
    // Operation will complete asynchronously.
    // Return std::noop_coroutine() to transfer execution to caller of
    // coroutine_handle::resume().
    return std::noop_coroutine();
  }

  // Operation completed synchronously.
  // Return current coroutine's handle to immediately resume
  // the current coroutine.
  return h;
}
```

Why have all three flavours?

So why do we still have the `void` and `bool` -returning flavours of `await_suspend()` when we have the symmetric-transfer flavour?

The reason is partly historical, partly pragmatic and partly performance.

The `void` -returning version could be entirely replaced by returning the `std::noop_coroutine_handle` type from `await_suspend()` as this would be an equivalent signal to the compiler that the coroutine is unconditionally transfering execution to the caller of `std::coroutine_handle::resume()` .

That it was kept was, IMO, partly because it was already in-use prior to the introduction of symmetric-transfer and partly because the `void` -form results in less-code/less-typing for the unconditional suspend case.

The `bool` -returning version, however, can have a slight win in terms of optimisability in some cases compared to the symmetric-transfer form.

Consider the case where we have a `bool` -returning `await_suspend()` method that is defined in another translation unit. In this case the compiler can generate code in the awaiting coroutine that will suspend the current coroutine and then conditionally resume it after the call to `await_suspend()` returns by just executing the next piece of code. It knows exactly the piece of code to execute next if `await_suspend()` returns `false` .

With the symmetric-transfer flavour we still need to represent the same outcomes; either return to the caller/resume or resume the current coroutine. Instead of returning `true` or `false` we need to return `std::noop_coroutine()` or the handle to the current coroutine. We can coerce both of these handles into a `std::coroutine_handle<void>` type and return it.

However, now, because the `await_suspend()` method is defined in another translation unit the compiler can't see what coroutine the returned handle is referring to and so when it resumes the coroutine it now has to perform some more expensive indirect calls and possibly some branches to resume the coroutine, compared to a single branch for the `bool` -returning case.

Now, it's possible that we might be able to get equivalent performance out of the symmetric transfer version one day. For example, we could write our code in such a way that `await_suspend()` is defined inline but calls a `bool` -returning method that is defined out-of-line and then conditionally returns the appropriate handle.

For example:

```
struct my_awaiter {
  bool await_ready();

  // Compilers should in-theory be able to optimise this to the same
  // as the bool-returning version, but currently don't do this optimisation.
  std::coroutine_handle<> await_suspend(std::coroutine_handle<> h) {
    if (try_start(h)) {
      return std::noop_coroutine();
    } else {
      return h;
    }
  }

  void await_resume();

private:
  // This method is defined out-of-line in a separate translation unit.
  bool try_start(std::coroutine_handle<> h);
}
```

However, current compilers (c. Clang 10) are not currently able to optimise this to as efficient code as the equivalent `bool` -returning version. Having said that, you're probably not going to notice the difference unless you're awaiting this in a really tight loop.

So, for now, the general rule is:

- If you need to unconditionally return to `.resume()` caller, use the `void` -returning flavour.
- If you need to conditionally return to `.resume()` caller or resume current coroutine use the `bool` -returning flavour.
- If you need to resume another coroutine use the symmetric-transfer flavour.

Rounding out

The new symmetric transfer capability added to coroutines for C++20 makes it much easier to write coroutines that recursively resume each other without fear of running into stack-overflow. This capability is key to making efficient and safe async coroutine types, such as the `task` one presented here.

This ended up being a much longer than expected post on symmetric transfer. If you made it this far, then thanks for sticking with it! I hope you found it useful.

In the next post, I'll dive into understanding how the compiler transforms a coroutine function into a state-machine.

Thanks

Thanks to Eric Niebler and Corentin Jabot for providing feedback on drafts of this post.